СХЕМЫ С ТАКТОВЫМ СИГНАЛОМ И СОСТОЯНИЕМ

Школа синтеза цифровых схем

ПРИ ПАРТНЕРСТВЕ

YADRO

Занятие №3

19 октября 2024

Школа синтеза цифровых схем

Школа синтеза цифровых схем

- Автор (разработчик материала) лекции — Силантьев Александр Михайлович

- Использование материалов и записи лекции и/или их частей без предварительного согласия не допускается.

- По вопросам использования материалов и записи лекции в коммерческих целях необходимо направить обращение в ООО «КНС ГРУПП» (YADRO) по адресу электронной почты synthesis@yadro.com.

- По вопросам некоммерческого использования материалов и записи лекции обращение может быть направлено в ООО «КНС ГРУПП» (YADRO) по адресу электронной почты synthesis@yadro.com, либо на адрес электронной почты автора Силантьева А.М. silantiev@org.miet.ru. Такое обращение обязательно должно содержать описание цели использования.

школа синтеза цифровых схем



Александр Силантьев

Руководитель лаборатории НИЛ ЭСК Университета МИЭТ

- Окончил МИЭТ в 2014 году
- 14-ти летний опыт инженерной деятельности в области проектирования IP-ядер и СнК.
- Старший преподаватель института МПСУ МИЭТ.
- Организатор первого в России инженерного хакатона по микроэлектронике и системам на кристалле.
- С 2014 года организатор семинаров, школ и олимпиад по популяризации электроники среди студентов и школьников

КОМБИНАЦИОННАЯ ЛОГИКА

школа синтеза цифровых схем

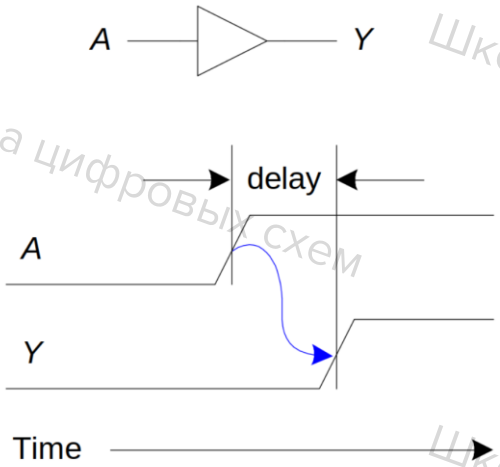


Школа синтеза цифровых схем

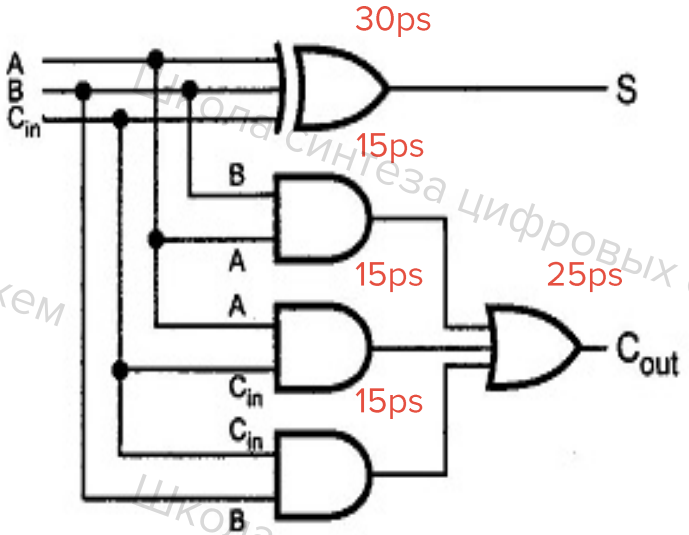
ШКОЛА СИНТЕЗА ЦИФРОВЫХ СХЕМ

ПРОБЛЕМА ВЫЧИСЛЕНИЙ В КОМБИНАЦИОННОЙ ЛОГИКЕ

Вычисления начинаются при изменении входных сигналов логики



Как понять когда результат на выходе комбинационной логики будет готов для дальнейших вычислений?

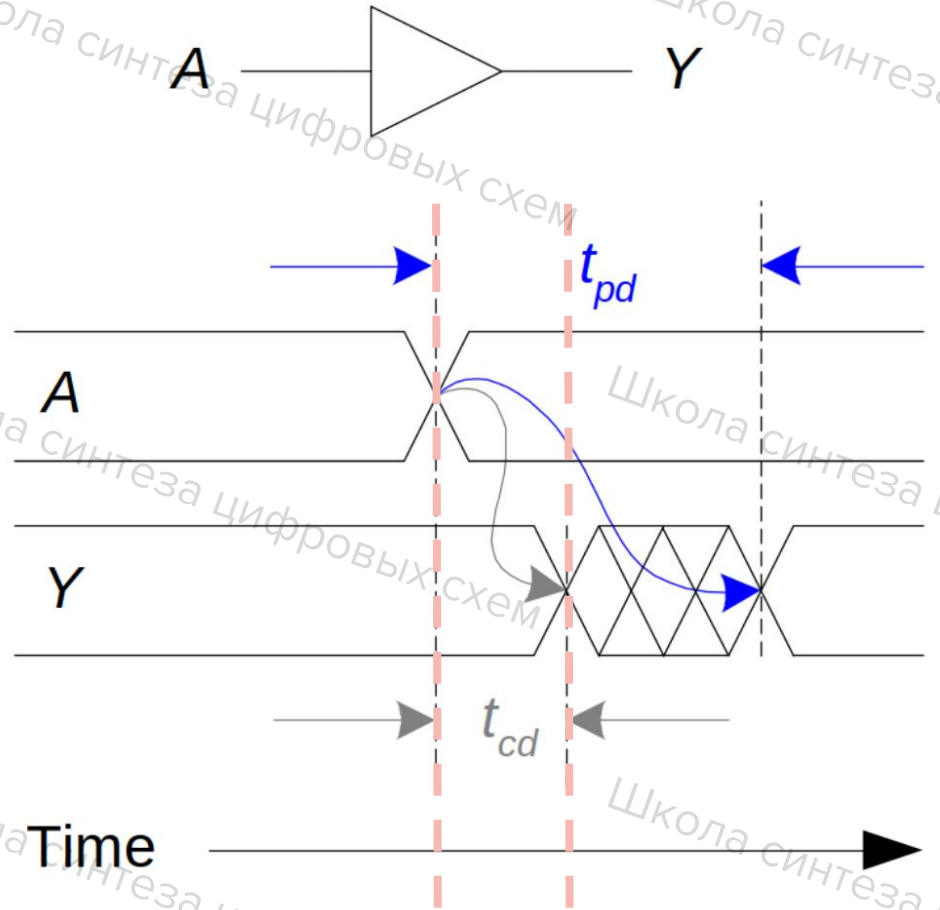


школа синтеза цифровых схем

CONTAMINATION AND PROPAGATION DELAYS

CONTAMINATION DELAY

Входы изменились, но на выходе результат пока что нестабильный

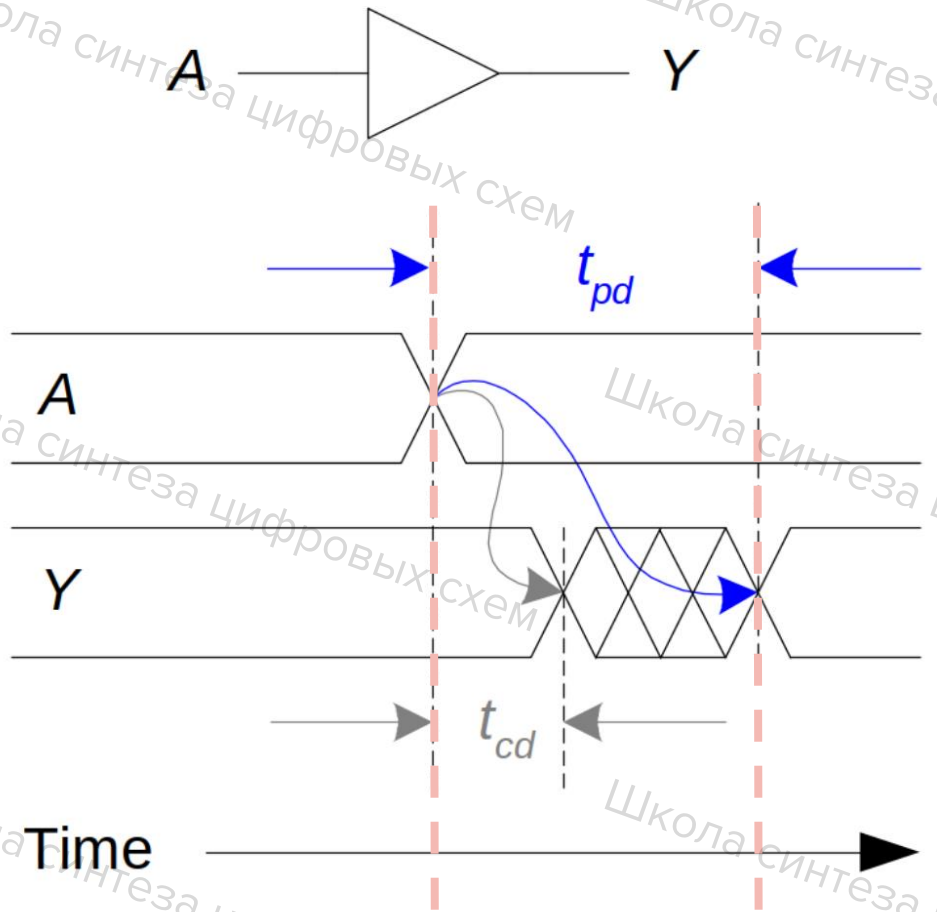


школа синтеза цифровых схем

CONTAMINATION AND PROPAGATION DELAYS

PROPAGATION DELAY

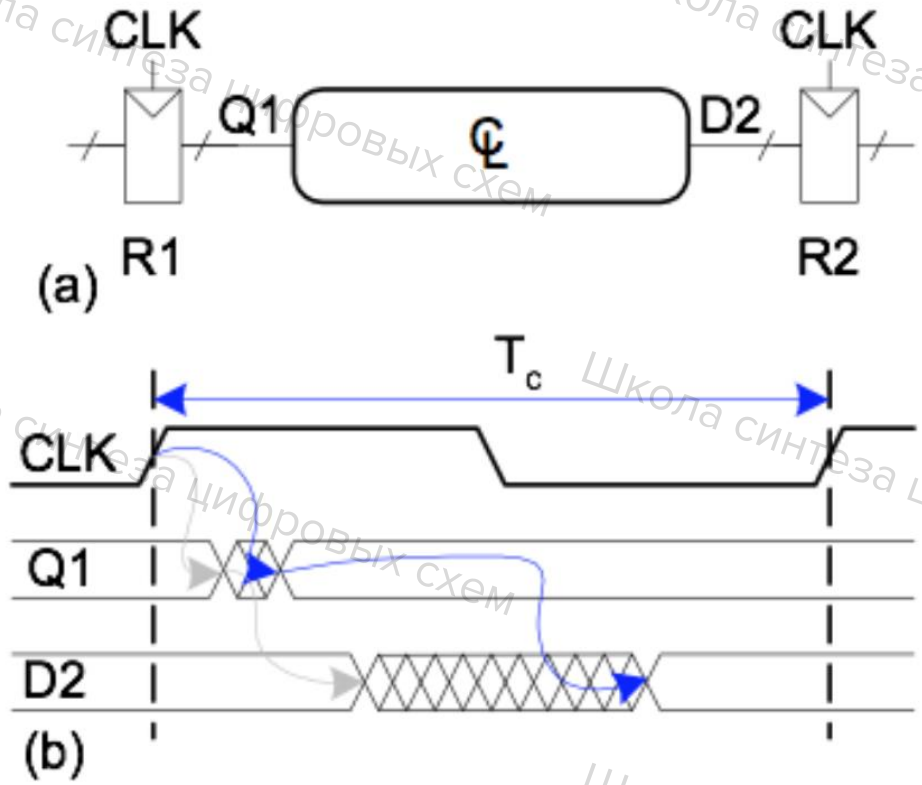
Стабильный результат на выходе комбинационной схемы



ИСПОЛЬЗОВАНИЕ ТАКТОВОГО СИГНАЛА

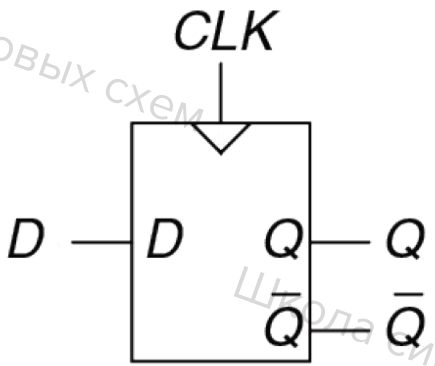
школа синтеза цифровых схем

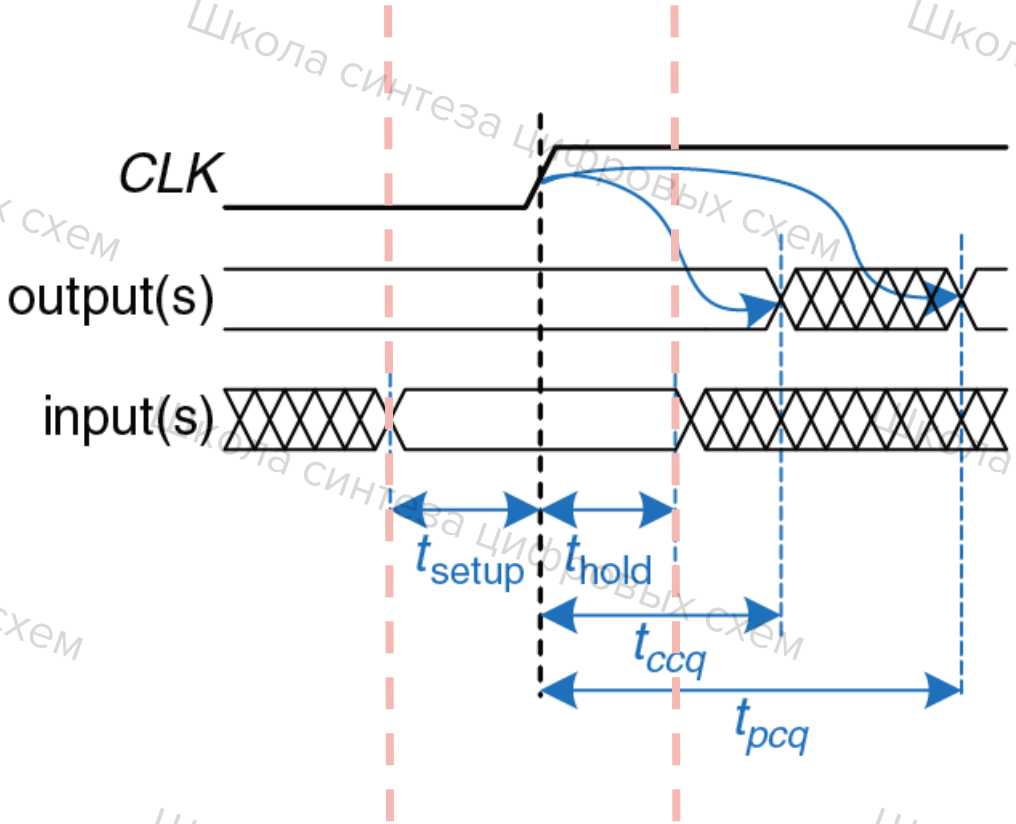
- Перед завершением вычислений выходные данные могут содержать случайные значения.
- Как логике узнать, когда результаты готовы и могут использоваться на следующем этапе вычислений?
- Вычисления можно синхронизировать с помощью специального сигнала — сигнала тактирования.



ВРЕМЕННЫЕ ХАРАКТЕРИСТИКИ

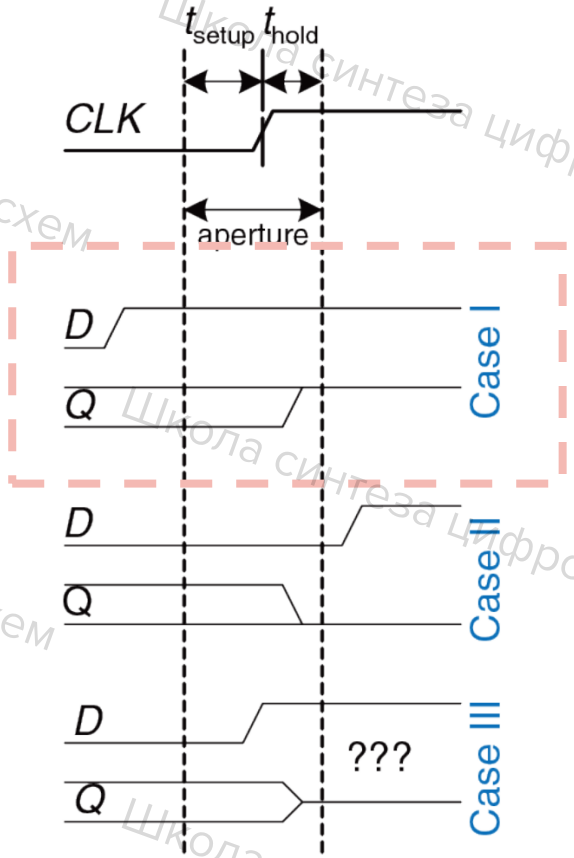
школа синтеза цифровых схем

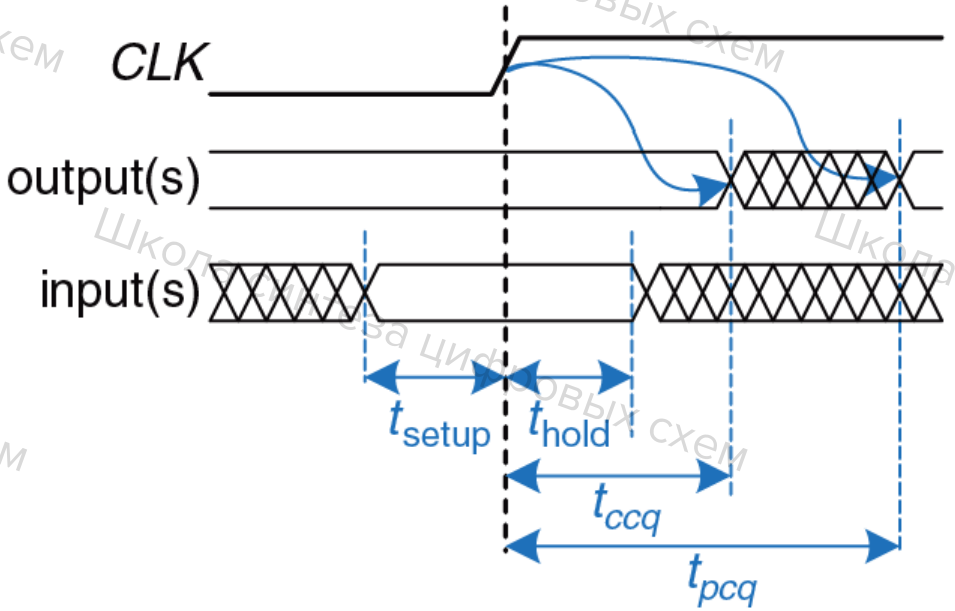




Книга Харрис & Харрис «Цифровая схемотехника и архитектура компьютера»

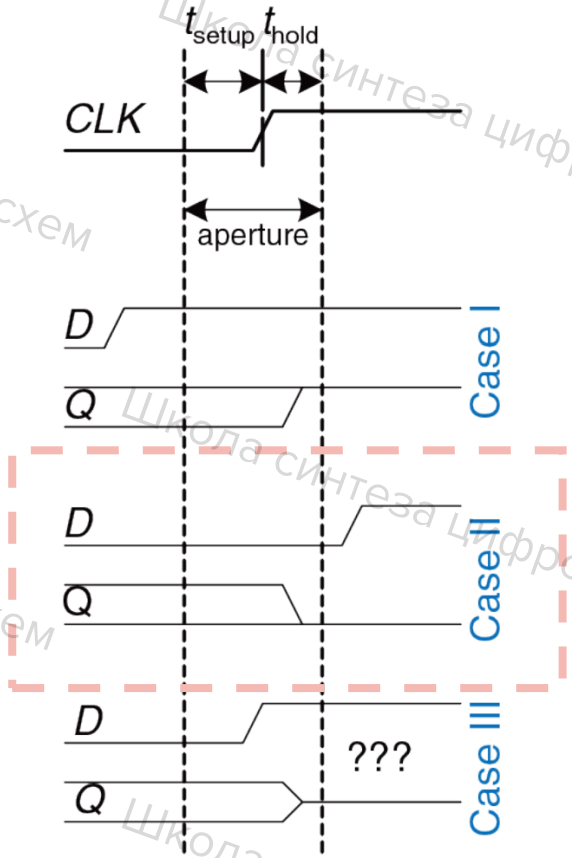
ВРЕМЕННЫЕ ХАРАКТЕРИСТИКИ. МЕТАСТАБИЛЬНОСТЬ

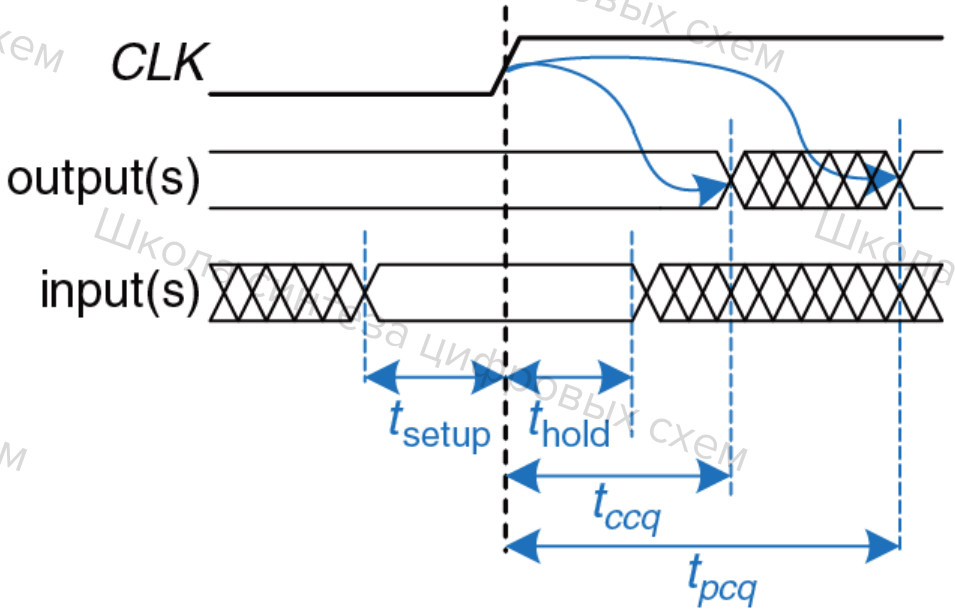




Книга Харрис & Харрис «Цифровая схемотехника и архитектура компьютера»

ВРЕМЕННЫЕ ХАРАКТЕРИСТИКИ. МЕТАСТАБИЛЬНОСТЬ

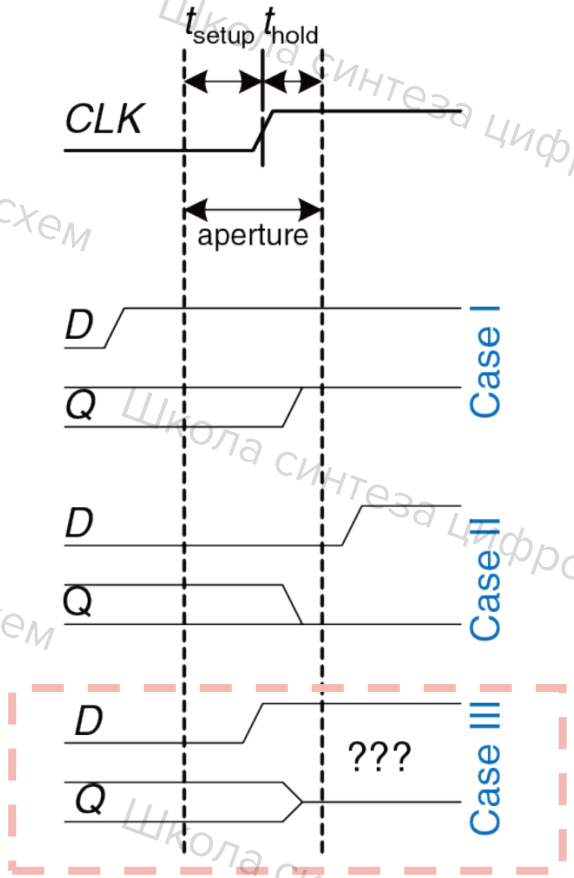


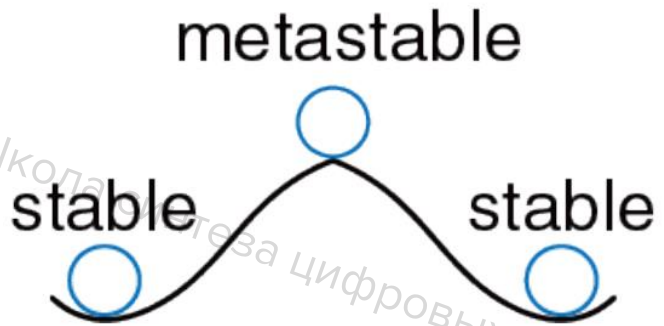


Книга Харрис & Харрис «Цифровая схемотехника и архитектура компьютера»

школа синтеза цифровых схем

ВРЕМЕННЫЕ ХАРАКТЕРИСТИКИ. МЕТАСТАБИЛЬНОСТЬ

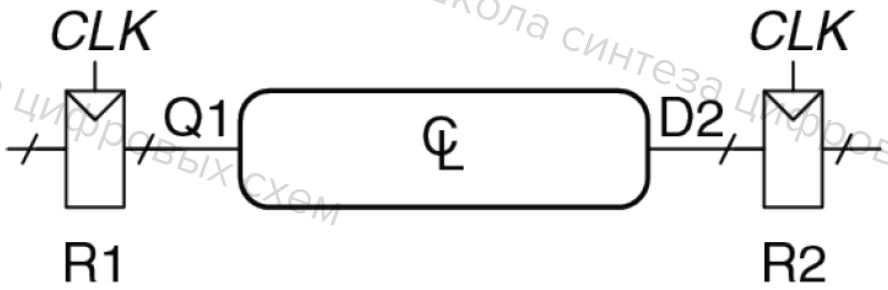


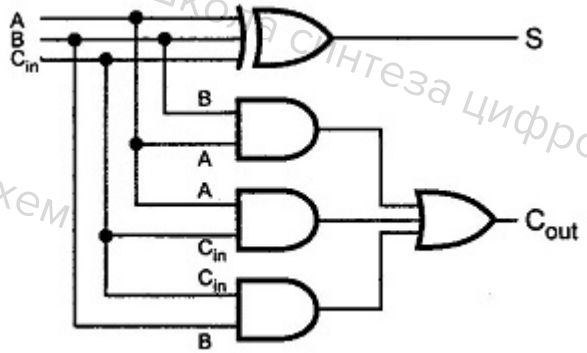


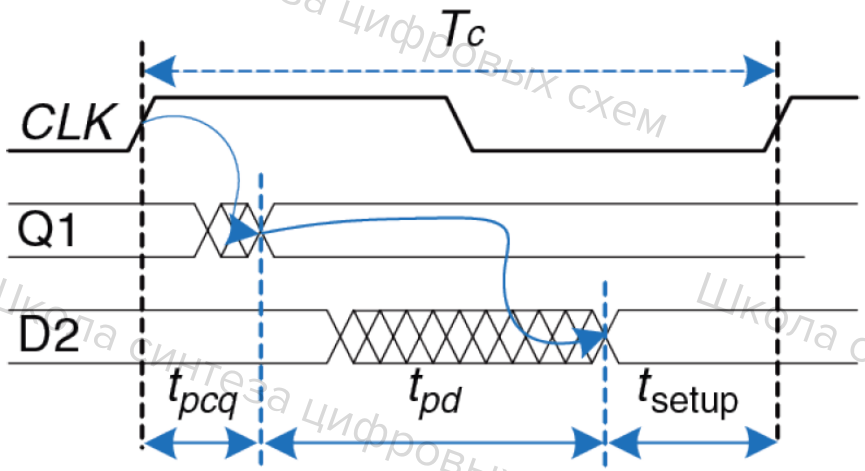
Книга Харрис & Харрис «Цифровая схемотехника и архитектура компьютера»

ВРЕМЕННЫЕ ХАРАКТЕРИСТИКИ.

школа синтеза цифровых схем





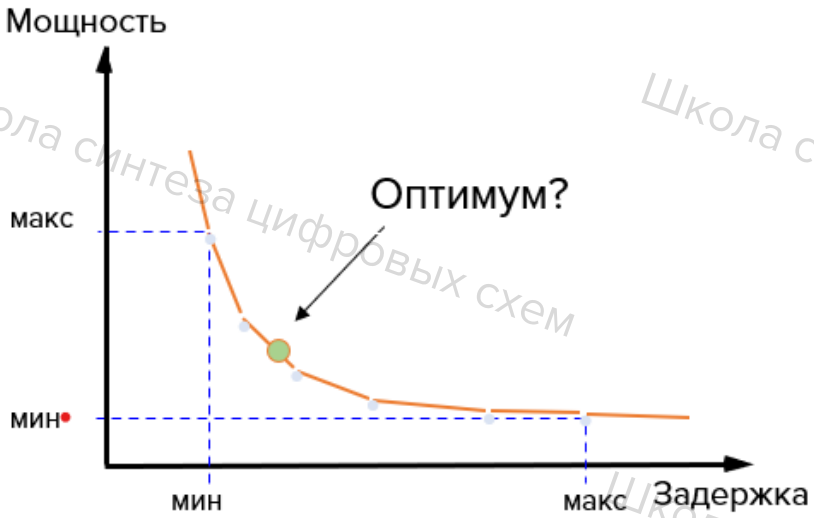


Книга Харрис & Харрис «Цифровая схемотехника и архитектура компьютера»

ЛОГИЧЕСКИЙ СИНТЕЗ

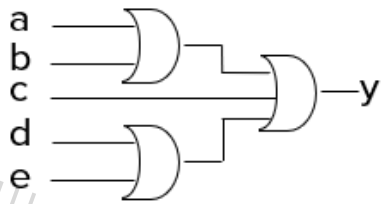
ШКОЛА СИНТЕЗА ЦИФРОВЫХ СХЕМ



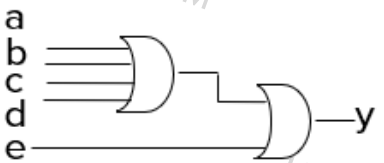


- Временная оптимизация
- Снижение энергопотребления
- Уменьшение площади

Пример: Y=a+b+c+d+e



Полная мощность: x6
Задержка: 2

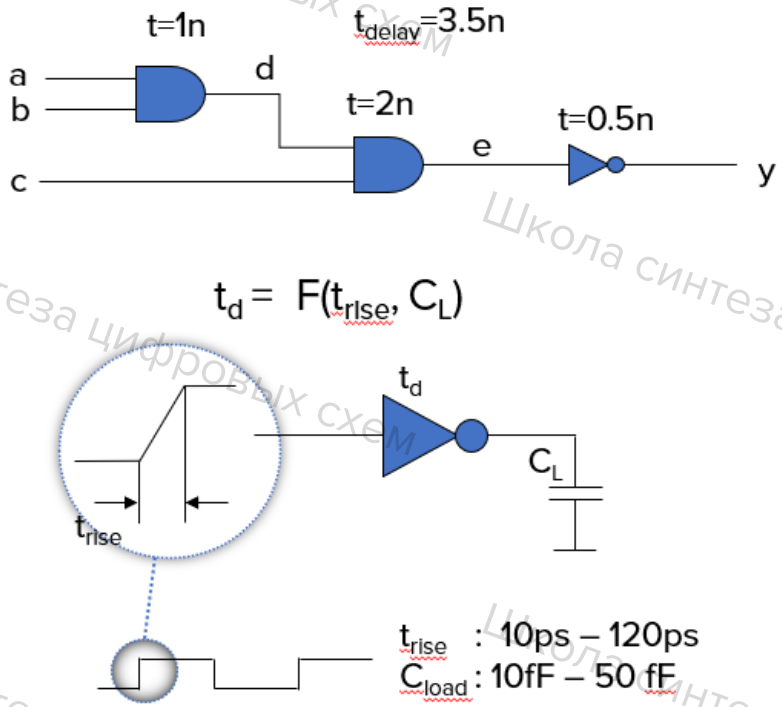


Полная мощность: x5
Задержка: 2.5

СТАТИЧЕСКИЙ ВРЕМЕННОЙ АНАЛИЗ

школа синтеза цифровых схем

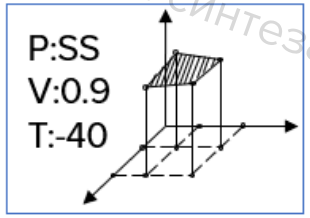
Задержка библиотечного элемента,
но в разных частях схемы — отличается

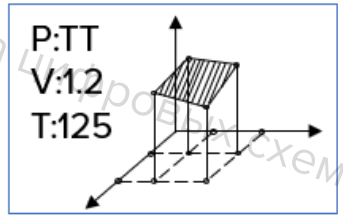


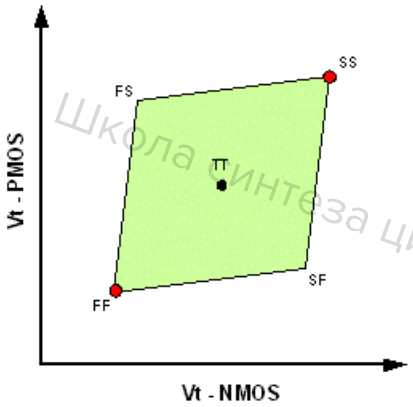
Задержка идентичного элемента
в разных экземплярах схемы – отличается

$t_d = f(PVT)$

Process: TT, FF, SS, etc.
Voltage: ∓10%
Temperature: -40…125°C





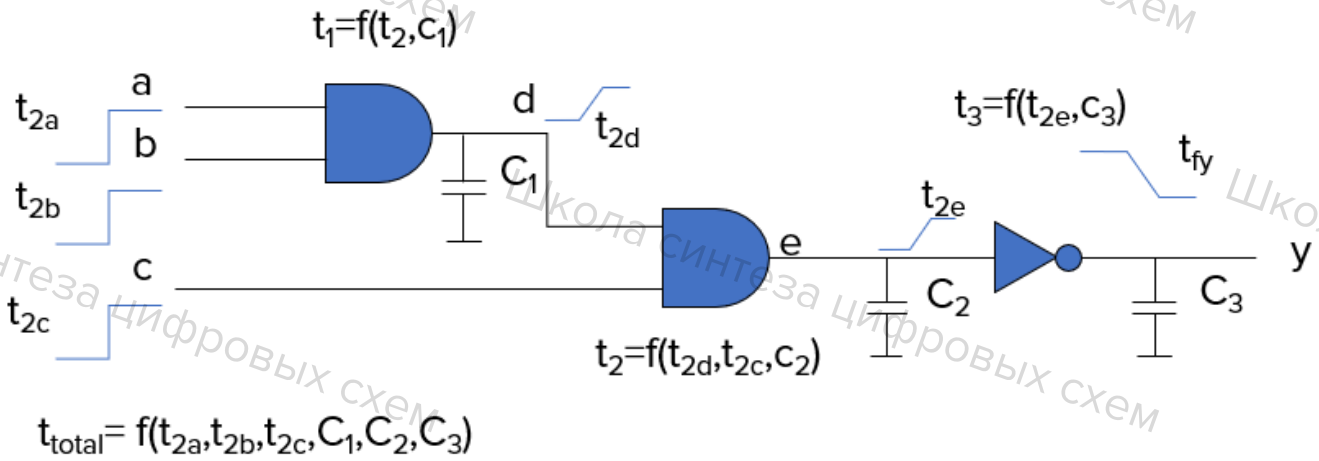


PVT-corner – «угол»
характеризации библиотеки

РАСЧЕТ ЗАДЕРЖКИ

школа синтеза цифровых схем

Для выбранного одиночного PVT:

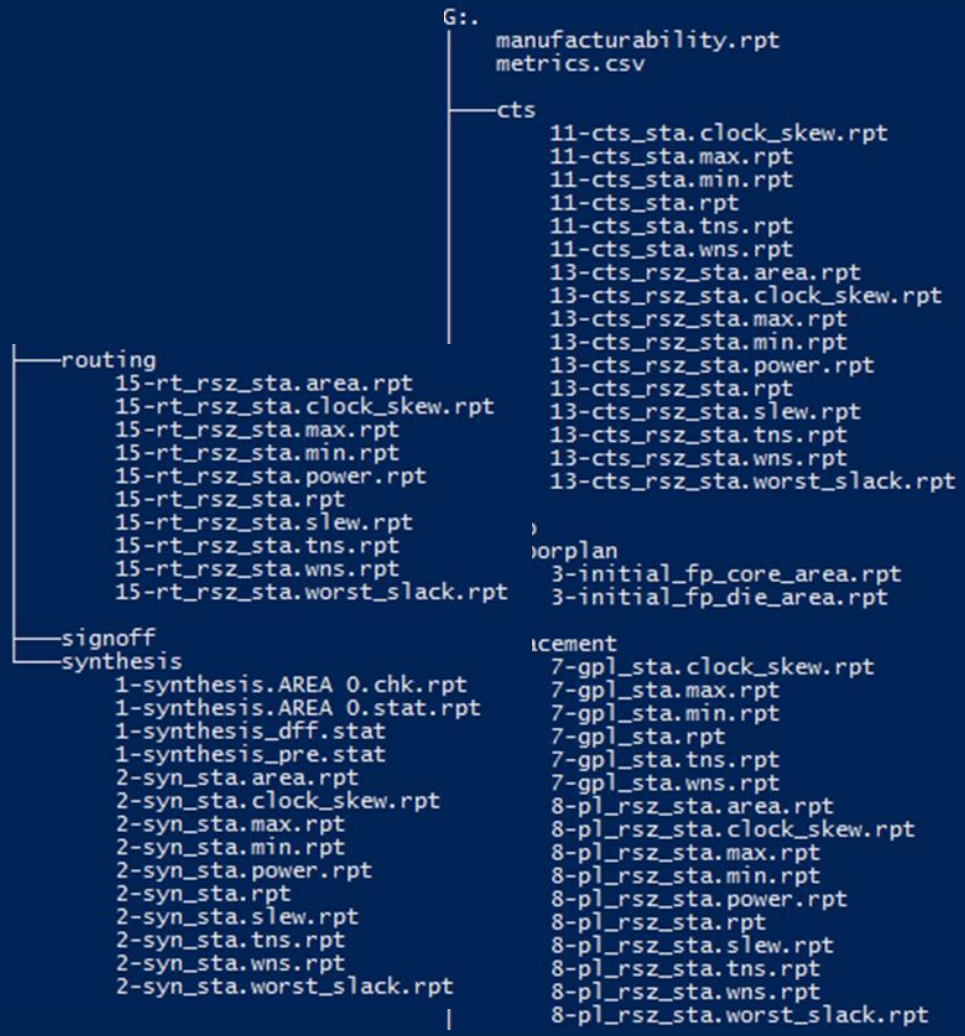


- Работоспособность схемы может быть гарантирована только при отсутствии нарушений временных ограничений для всех путей и PVT углов!
- Ограничения определяются параметрами синхросигнала и условиями внешней логики;
- Минимальный набор: worst case (WC) для проверки SETUP / best case (BC) для проверки HOLD.

ВРЕМЕННЫЕ ОТЧЕТЫ (STA REPORTS)

ШКОЛА СИНТЕЗА ЦИФРОВЫХ СХЕМ

- STA проводится на всех основных этапах, встроен в процесс оптимизации синтезаторов;
- Основные метрики: WNS/TNS — worst/total negative slack;
- slack - запас на выполнения ограничения;
- Частота 1 ГГц / задержка 0,999 нс: slack = 1 пс;
- Одновременно анализируется мощность, площадь;
- Анализ проводится для нескольких PVT-углов;
- Цель: неотрицательный slack с наилучшими Area/Power.



СТРУКТУРА ОТЧЕТА

ШКОЛА СИНТЕЗА ЦИФРОВЫХ СХЕМ

Path type – max – проверка setup

Fanout – число подключенных элементов к выходу ячейки

Cap – нагрузочная емкость

Slew – время переключения

Delay – задержка

Time – накопленная задержка

```
Startpoint: _672_ (rising edge-triggered flip-flop clocked by core_clock)
Endpoint: _702_ (rising edge-triggered flip-flop clocked by core_clock)
Path Group: core_clock
Path Type: max

Fanout     Cap    Slew   Delay    Time   Description
---------------------------------------------------------
                  0.00    0.00    0.00   clock core_clock (rise edge)
                          0.00    0.00   clock network delay (ideal)
                  0.00    0.00    0.00 ^ _672_/CK (DFF_X1)
                  0.24    0.32    0.32 ^ _672_/Q (DFF_X1)
     41  102.33                          req_rdy (net)
                  0.24    0.00    0.32 ^ _585_/A2 (NOR2_X1)
                  0.08    0.11    0.43 v _585_/ZN (NOR2_X1)
     16   31.08                          _240_ (net)
                  0.08    0.00    0.43 v _646_/A1 (AND2_X1)
                  0.01    0.06    0.49 v _646_/ZN (AND2_X1)
      1    1.85                          _290_ (net)
                  0.01    0.00    0.49 v _647_/A (AOI21_X1)
                  0.04    0.04    0.53 ^ _647_/ZN (AOI21_X1)
      1    1.97                          _291_ (net)
                  0.04    0.00    0.53 ^ _648_/A2 (NAND2_X1)
                  0.01    0.02    0.54 v _648_/ZN (NAND2_X1)
      1    1.37                          _031_ (net)
                  0.01    0.00    0.54 v _702_/D (DFF_X1)
                                  0.54   data arrival time

                  0.00    0.46    0.46   clock core_clock (rise edge)
                          0.00    0.46   clock network delay (ideal)
                          0.00    0.46   clock reconvergence pessimism
                                  0.46 ^ _702_/CK (DFF_X1)
                         -0.04    0.42   library setup time
                                  0.42   data required time
---------------------------------------------------------
                                  0.42   data required time
                                 -0.54   data arrival time
---------------------------------------------------------
                                 -0.12   slack (VIOLATED)
```

В отчете 2 величины:

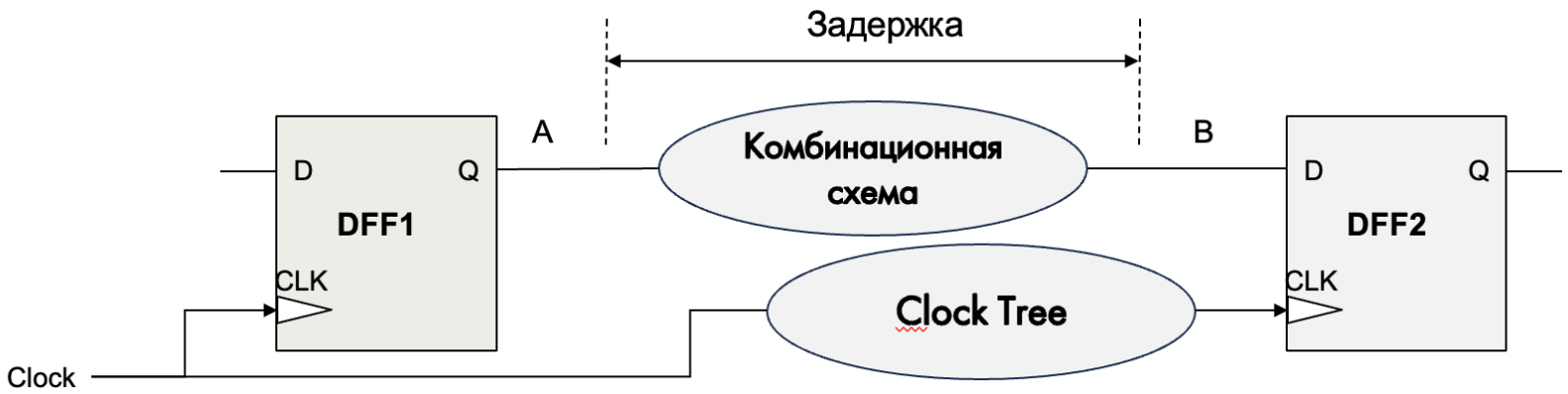
- Arrival Time – фактическое прохождение сигнала по схеме;
- Определяется задержкой вентилей и цепей + [внешнее ограничение];
- Required Time требуемое:
- T – setup – [output] – [uncertainty].

БЫСТРОДЕЙСТВИЕ И ОГРАНИЧЕНИЯ

ШКОЛА СИНТЕЗА ЦИФРОВЫХ СХЕМ

Основное ограничение

- В синхронной схеме, сигналы на входах регистров должны удовлетворять требованиям по setup/hold.



Задержка (tpd) > T_{clock} ,
то нарушено требование setup
не соблюдается в точке B

кратчайший путь (tcd) < t_{hold} , то
нарушено требование hold в DFF2.

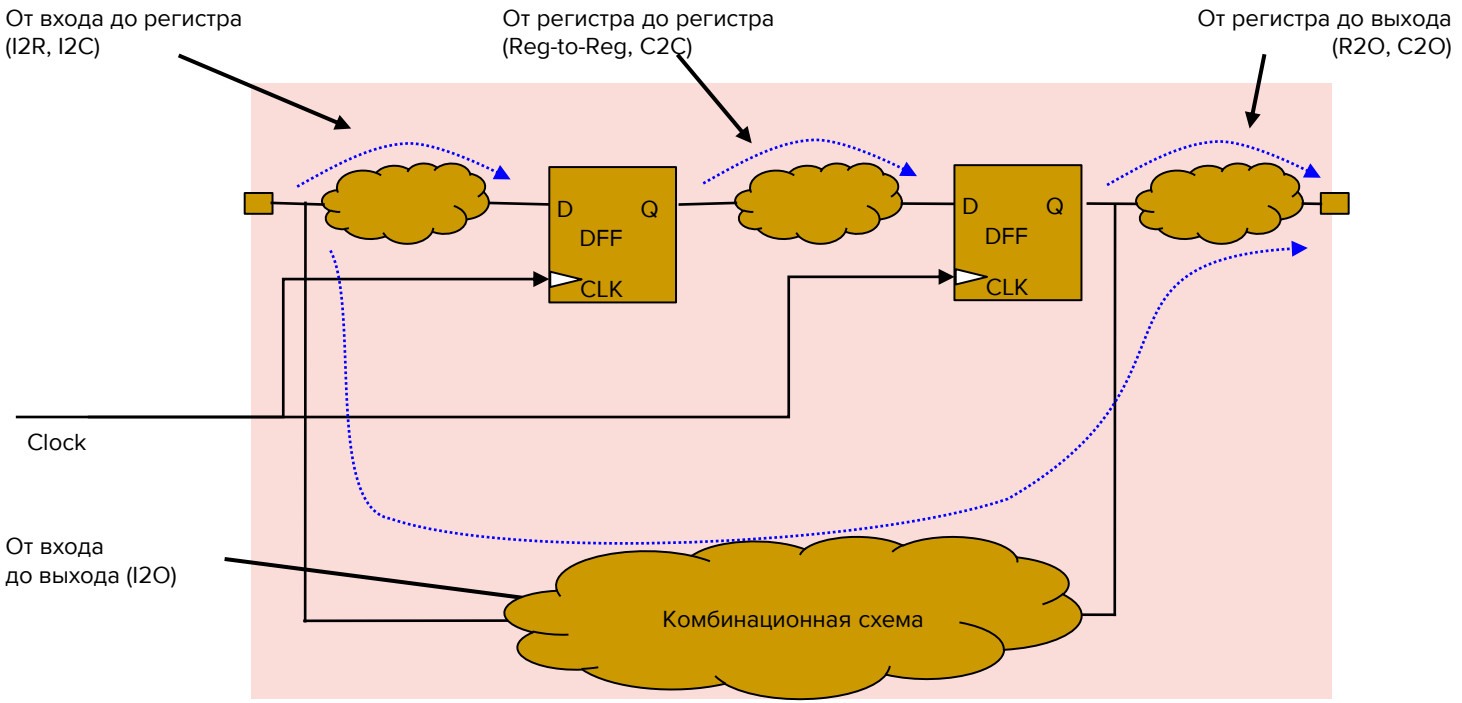
Книга Харрис & Харрис — раздел 3.5!

ТИПЫ ПУТЕЙ

ШКОЛА СИНТЕЗА ЦИФРОВЫХ СХЕМ

Основные виды временных путей (timing paths):

- Startpoint — входной порт, выходной пин (Q) триггера(регистра);
- Endpoint — выходной порт, входной пин (D) триггера(регистра).

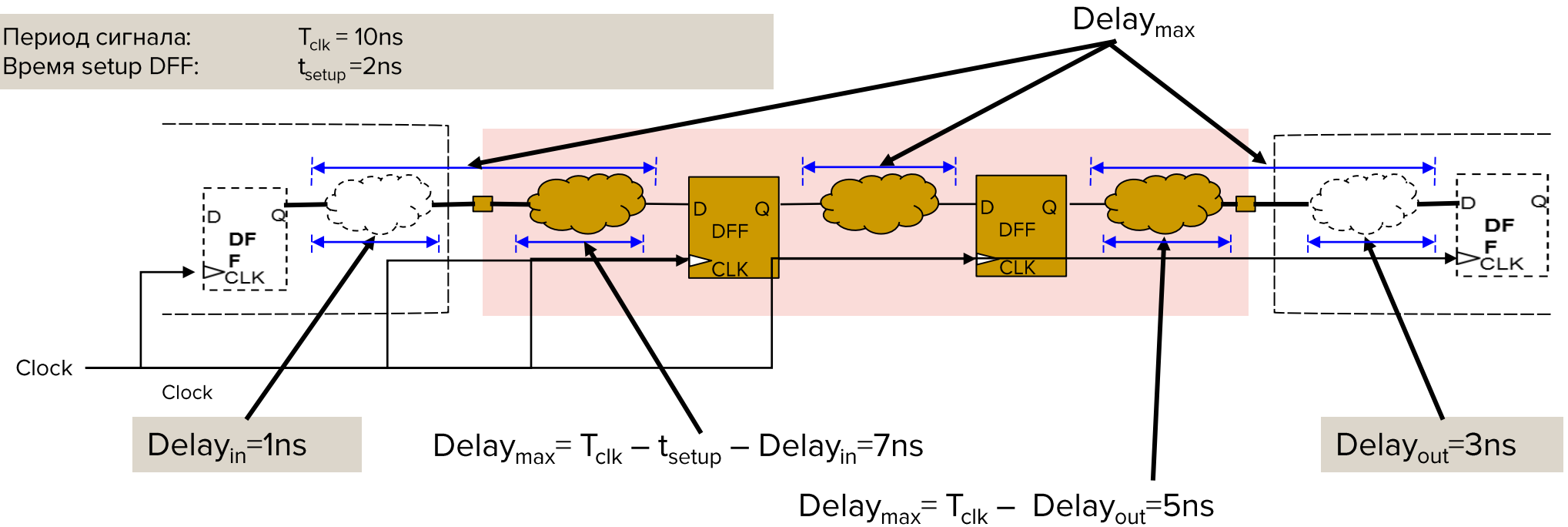


ПРИМЕР ОСНОВНЫХ DESIGN CONSTRAINTS

школа синтеза цифровых схем

```
create_clock -period 10 [get_ports Clk]
set_input_delay -max 1 -clock Clk [get_ports IN]
set_output_delay -max 3 -clock Clk [get_ports OUT]
```

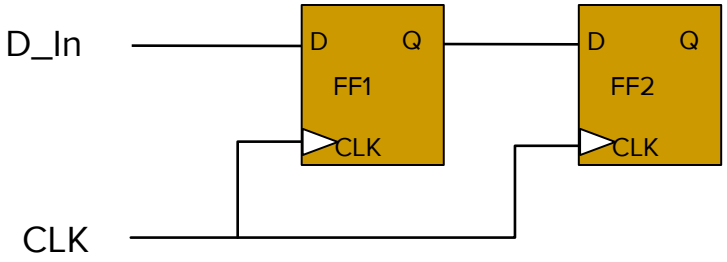
Период сигнала: T_{clk} = 10ns
Время setup DFF: t_{setup} =2ns



ДЕРЕВО ТАКТОВОГО СИГНАЛА

школа синтеза цифровых схем

- На этапе логического синтеза тактовый сигнал — идеален (одновременно приходит на триггера, форма стабильна);
- Дерево синхронизации строится на этапе топологического проектирования;
- Необходима модель эффектов задержки тактового сигнала.



Логическая схема

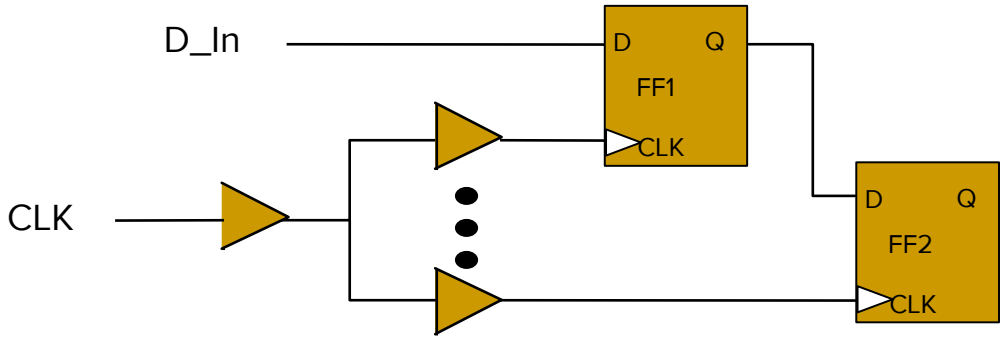
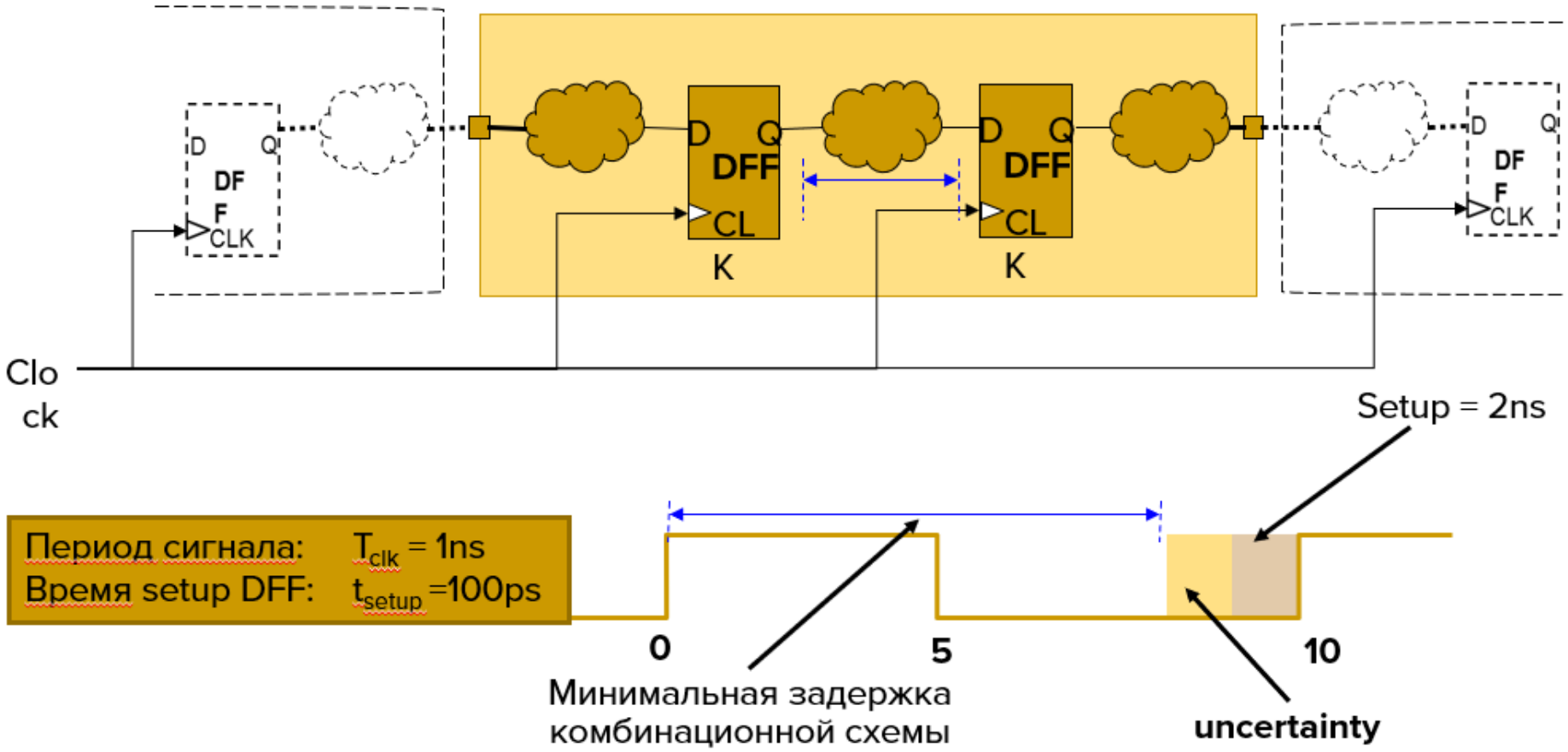


Схема после синтеза топологии

школа синтеза цифровых схем

ВЛИЯНИЕ UNCERTAINTY НА РЕЗУЛЬТАТЫ STA



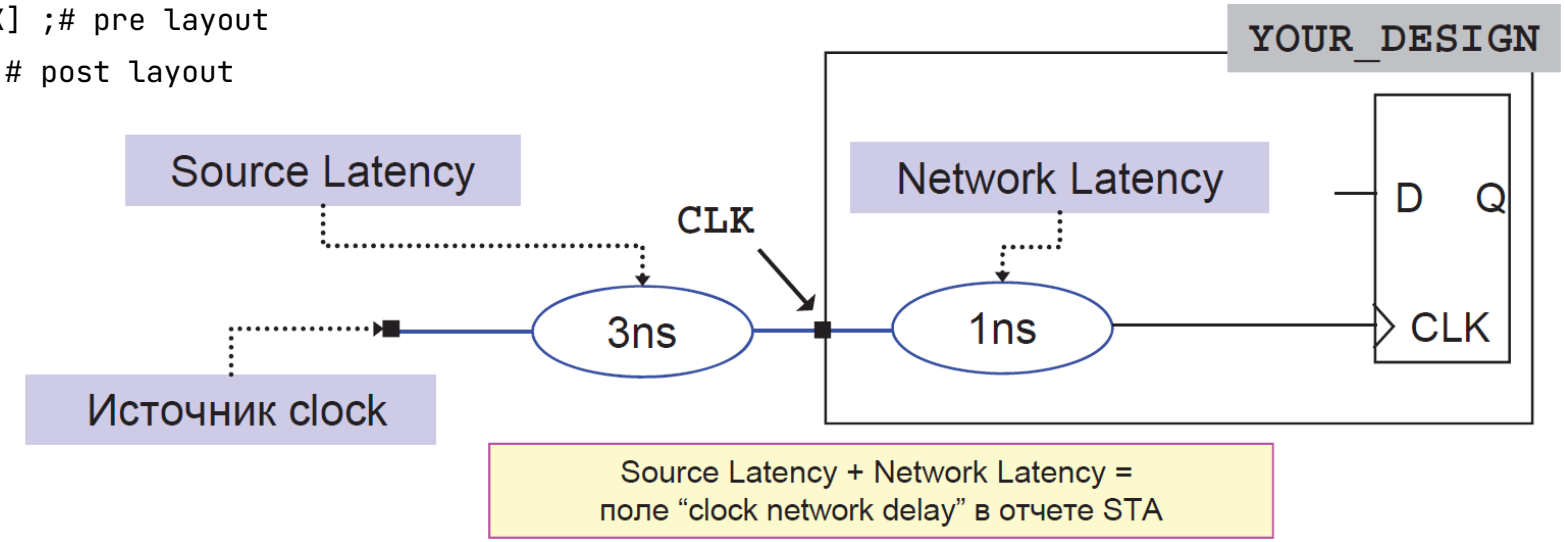

```
set_clock_uncertainty -setup 0.15  [get_clocks CLK]
```

УСТАНОВКА CLOCK LATENCY

ШКОЛА СИНТЕЗА ЦИФРОВЫХ СХЕМ

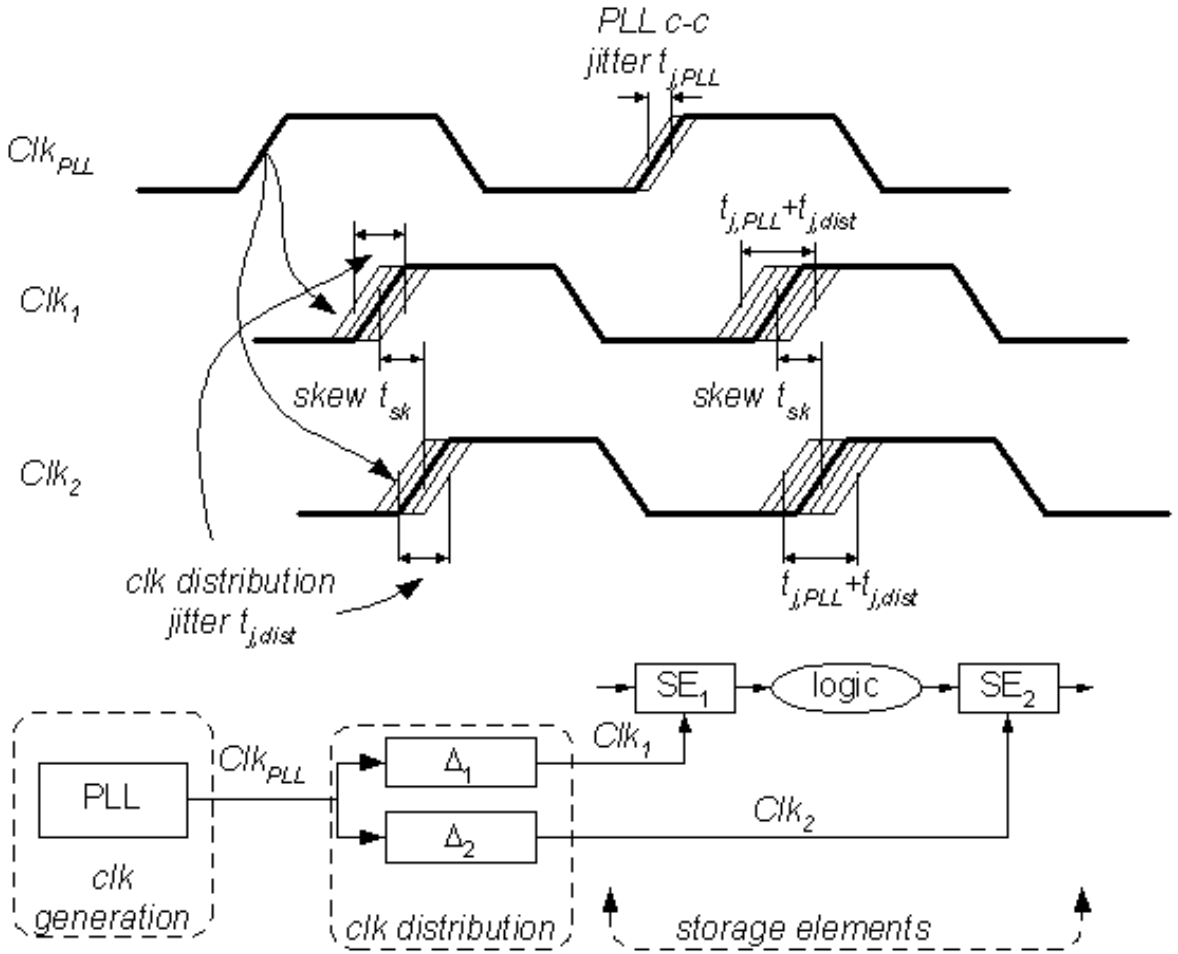
- Задержка источника (source latency): задержка тактового сигнала от его источника до входных портов схемы (полезна когда устройство генерации синхроимпульсов не является частью схемы);
- Задержка сети (network latency): задержка распространения тактового сигнала внутри схемы.

```
create_clock -period 10 [get_ports CLK]
set_clock_latency -source -max 3 [get_clocks CLK]
set_clock_latency -max 1 [get_clocks CLK] ;# pre layout
# set_propagated_clock [get_ports CLK] ;# post layout
```



ВЕЛИЧИНА UNCERTAINTY

ШКОЛА СИНТЕЗА ЦИФРОВЫХ СХЕМ

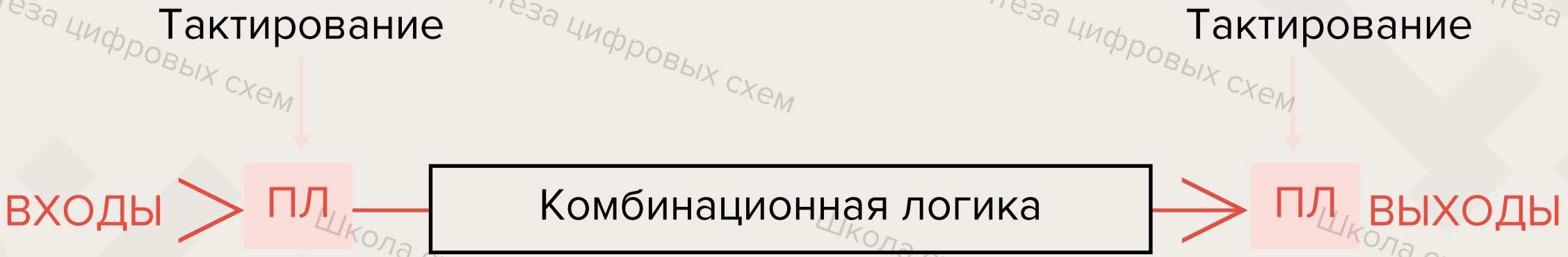


- Skew (разфазировка) – разность фаз синхросигнала на триггерах в одном временном пути
- Jitter (джиттер) – нестабильность периода синхросигнала, определяемая генератором

  Min. uncertainty = jitter + skew
- margin (запас) — ужесточение ограничения, проектный резерв для компенсации негативных эффектов

ПОСЛЕДОВАТЕЛЬНОСТНАЯ ЛОГИКА

ШКОЛА СИНТЕЗА ЦИФРОВЫХ СХЕМ



Школа синтеза цифровых схем

ВРЕМЕННЫЕ ДИАГРАММЫ

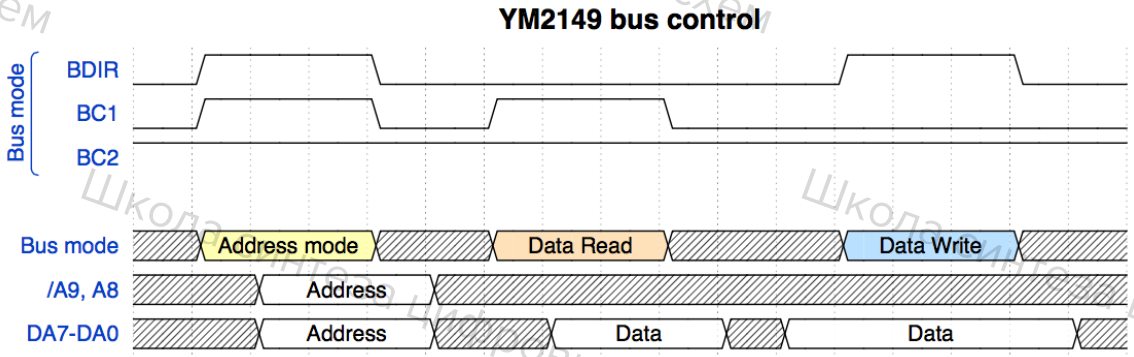


Школа синтеза цифровых схем

ВРЕМЕННЫЕ ДИАГРАММЫ

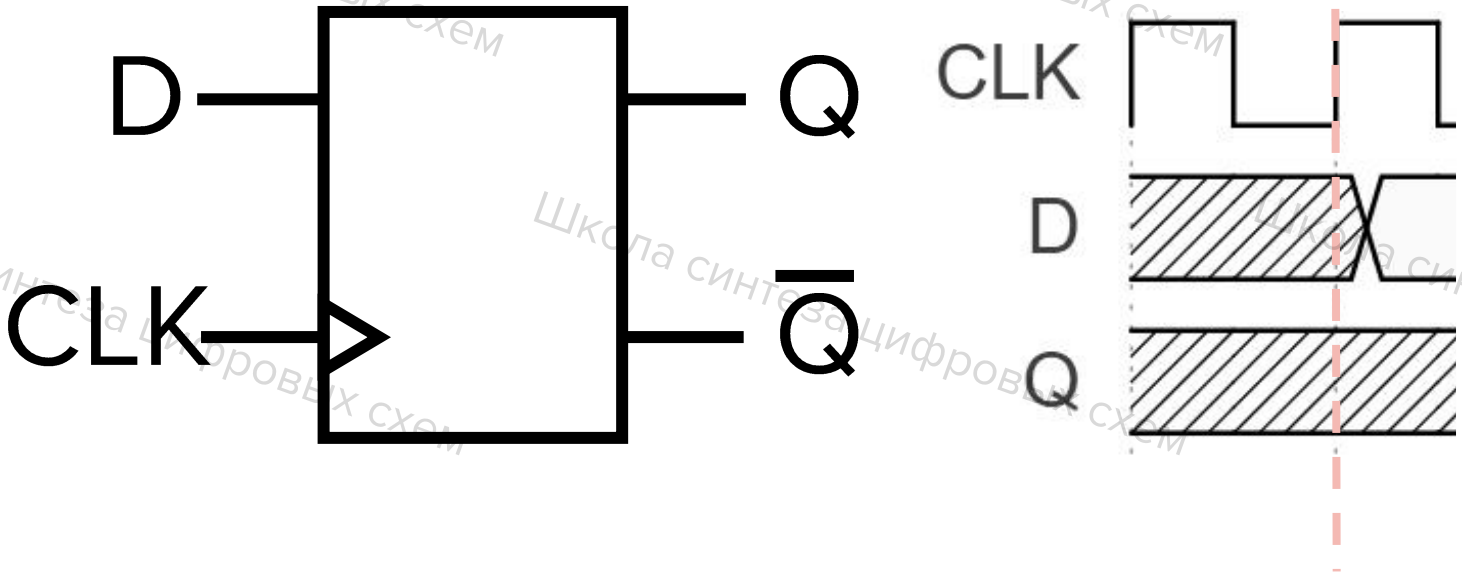
ШКОЛА СИНТЕЗА ЦИФРОВЫХ СХЕМ





D-ТРИГГЕР

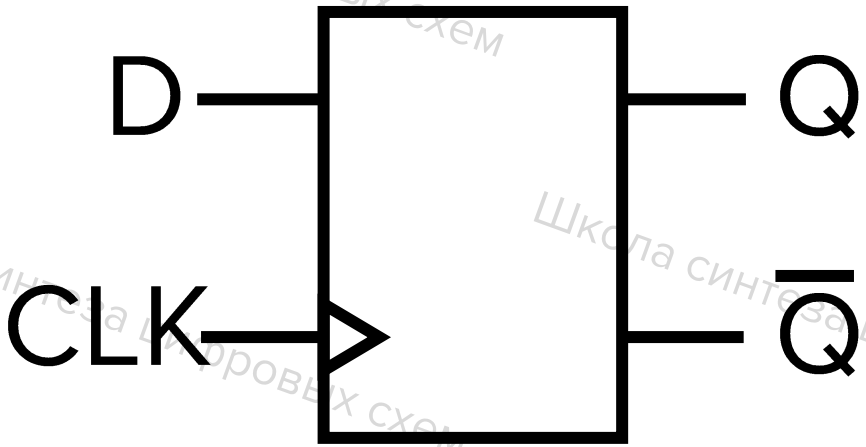
школа синтеза цифровых схем

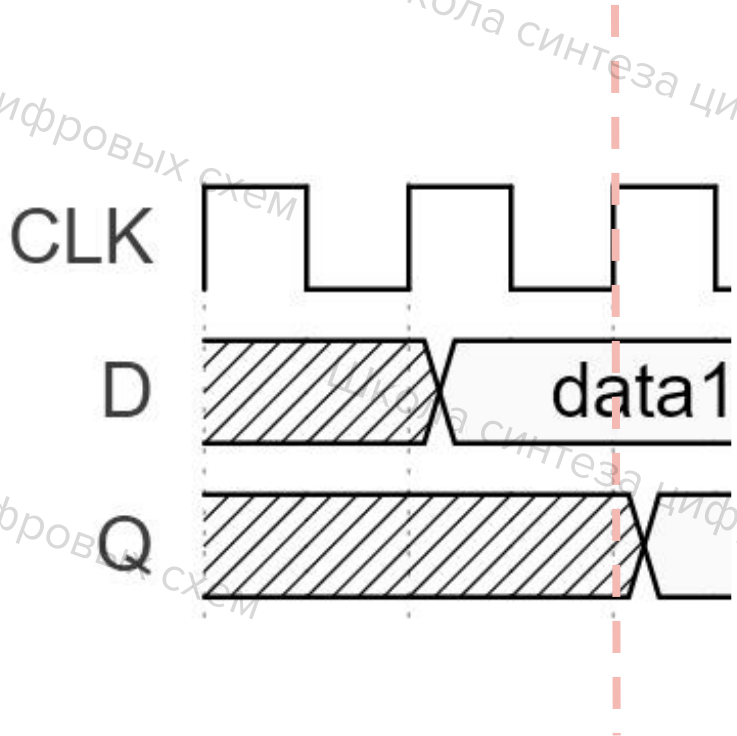


Школа синтеза цифровых схем

D-ТРИГГЕР

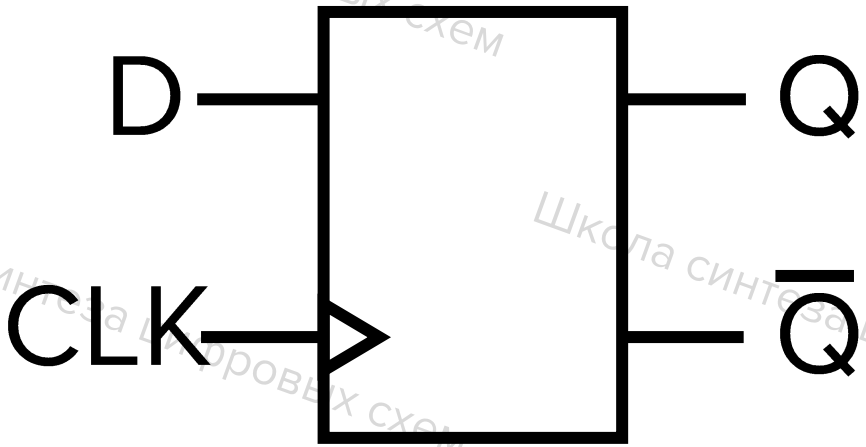
школа синтеза цифровых схем

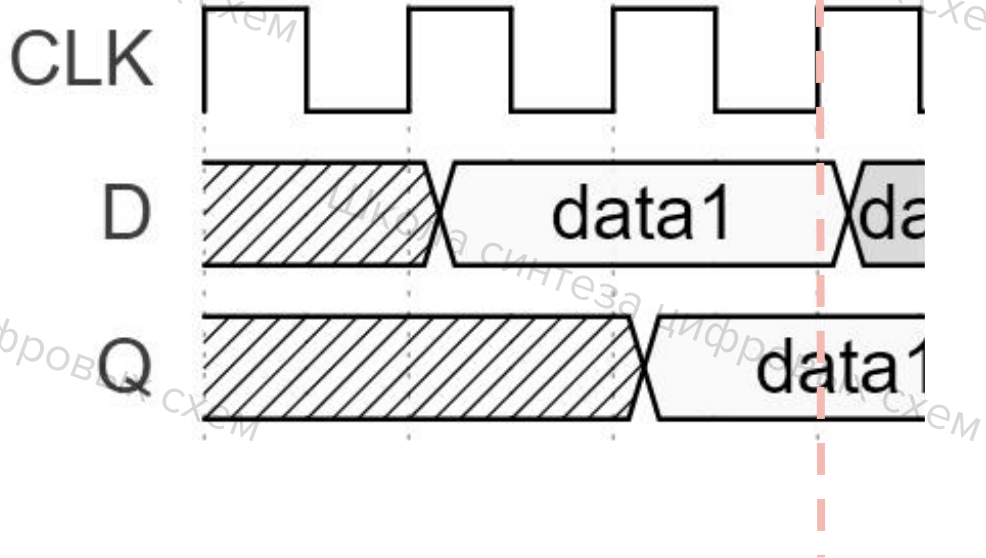




D-ТРИГГЕР

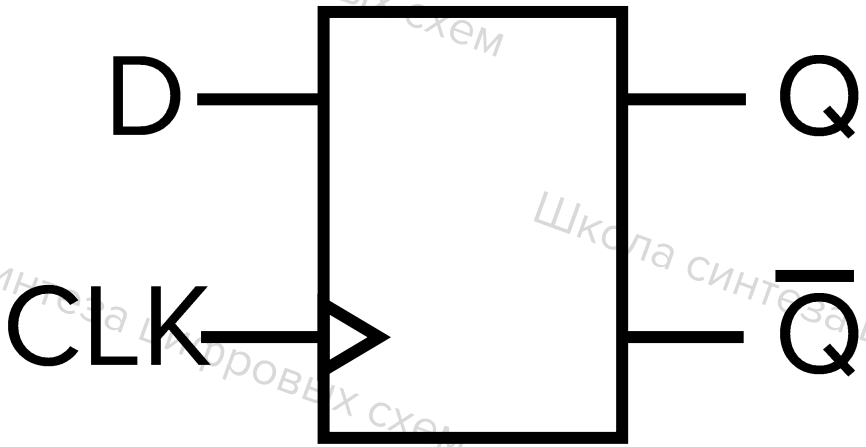
школа синтеза цифровых схем

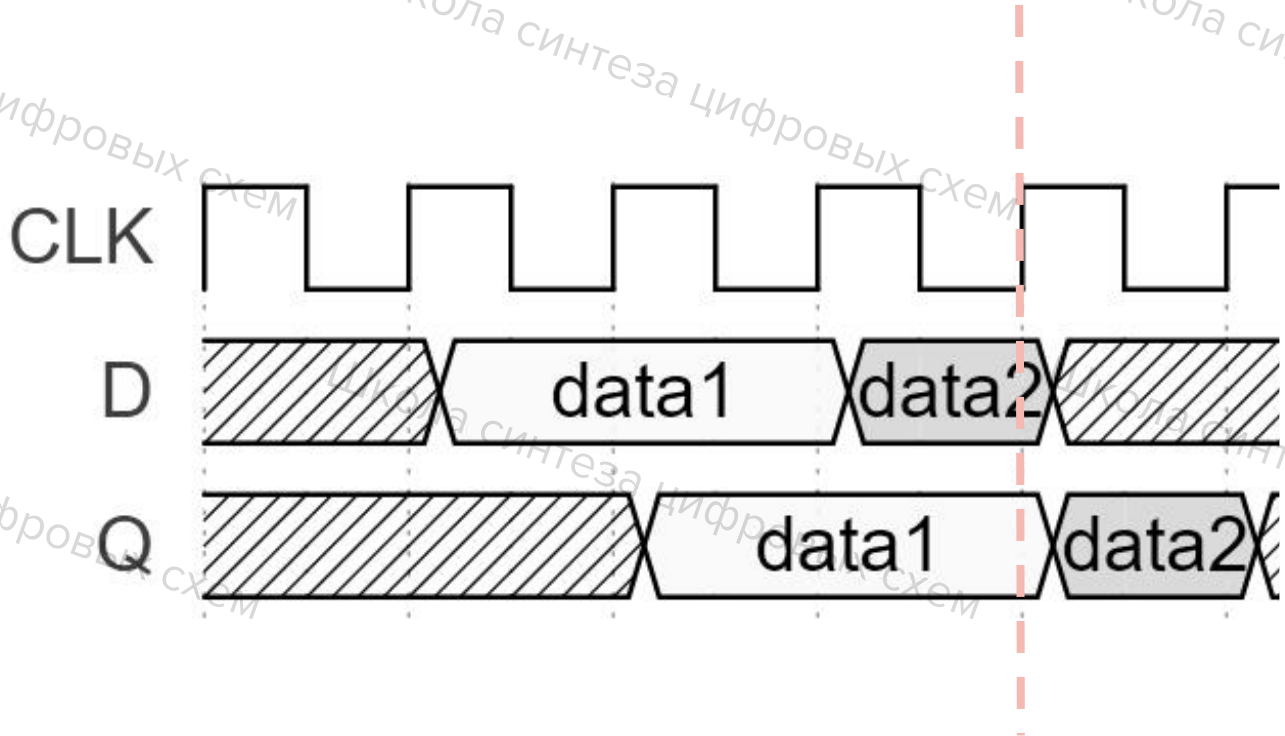




D-ТРИГГЕР

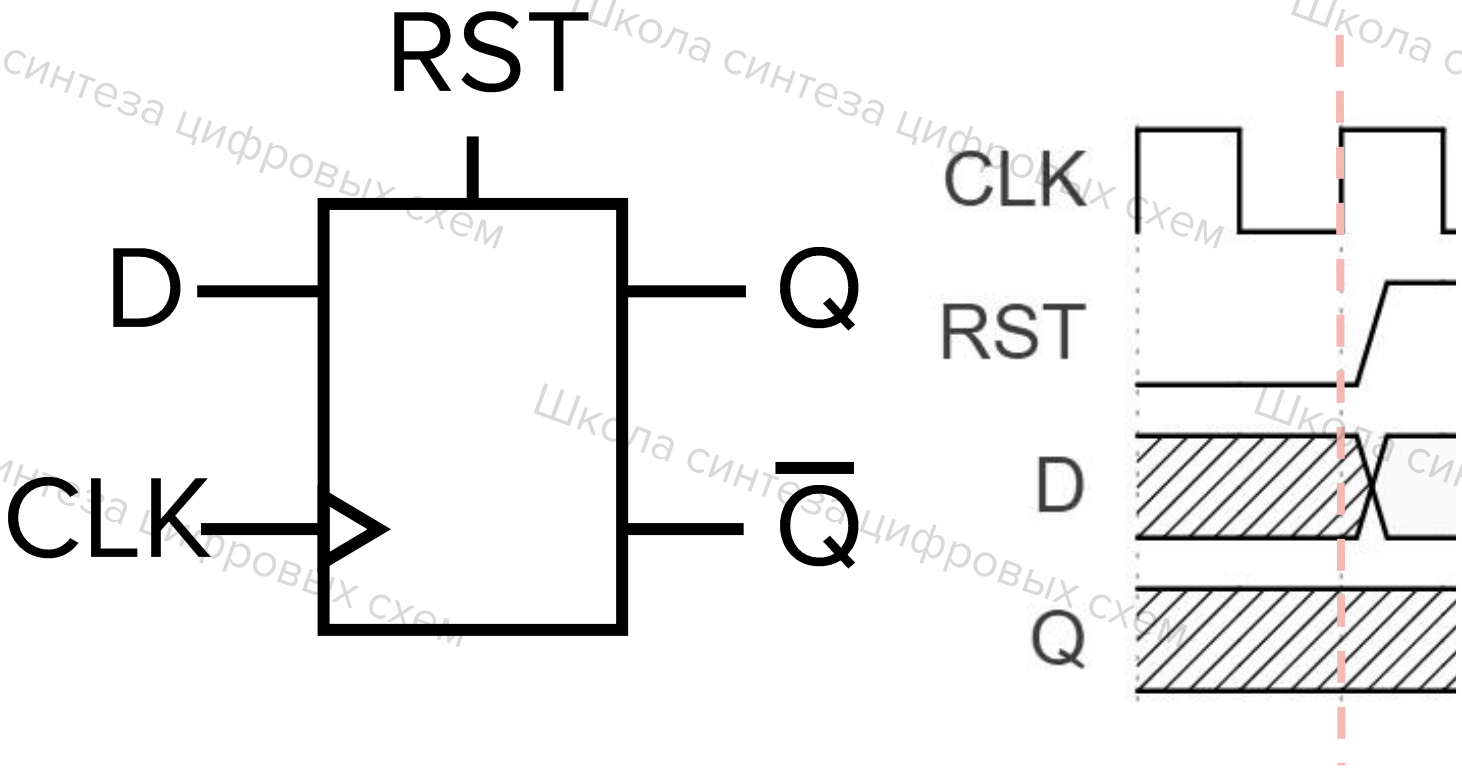
ШКОЛА СИНТЕЗА ЦИФРОВЫХ СХЕМ





D-ТРИГГЕР. СИГНАЛ СБРОСА

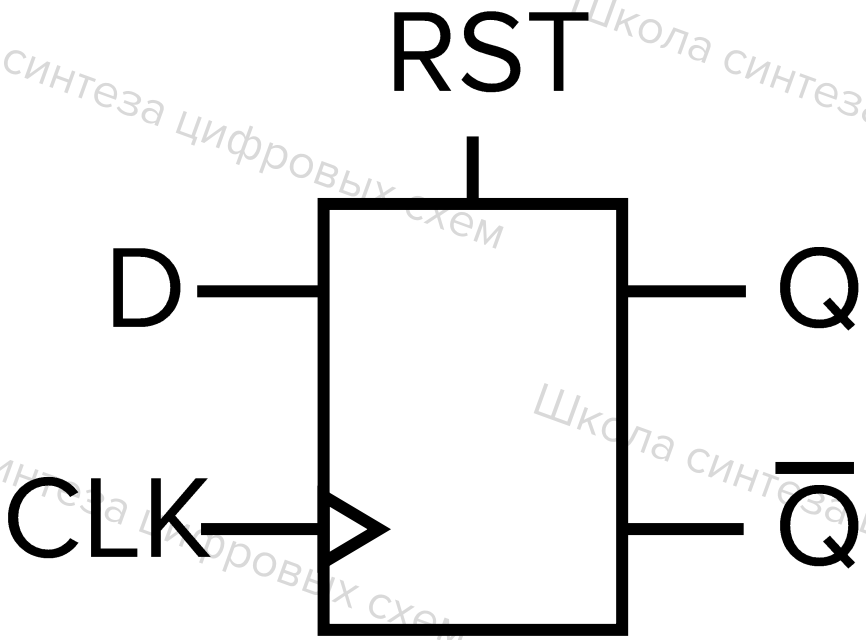
школа синтеза цифровых схем

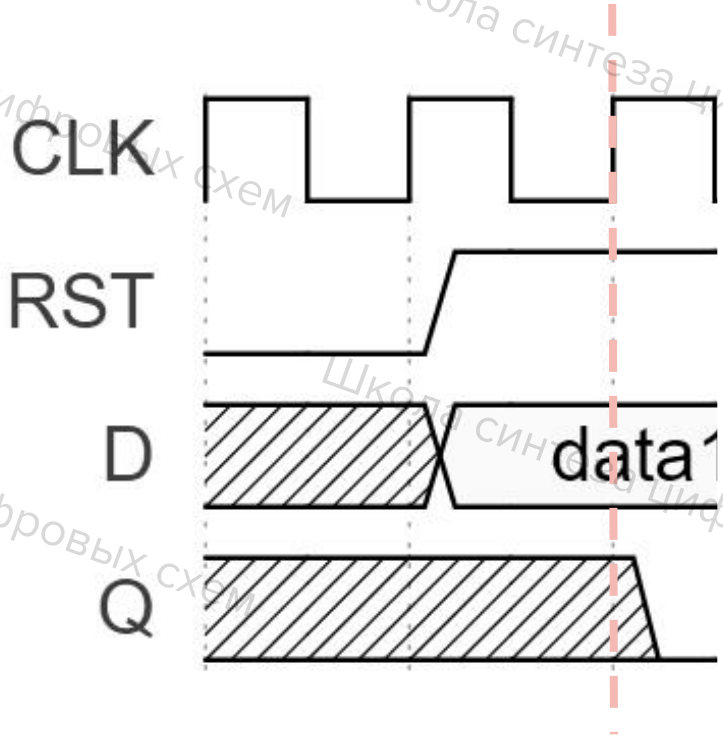


Школа синтеза цифровых схем

D-ТРИГГЕР. СИГНАЛ СБРОСА

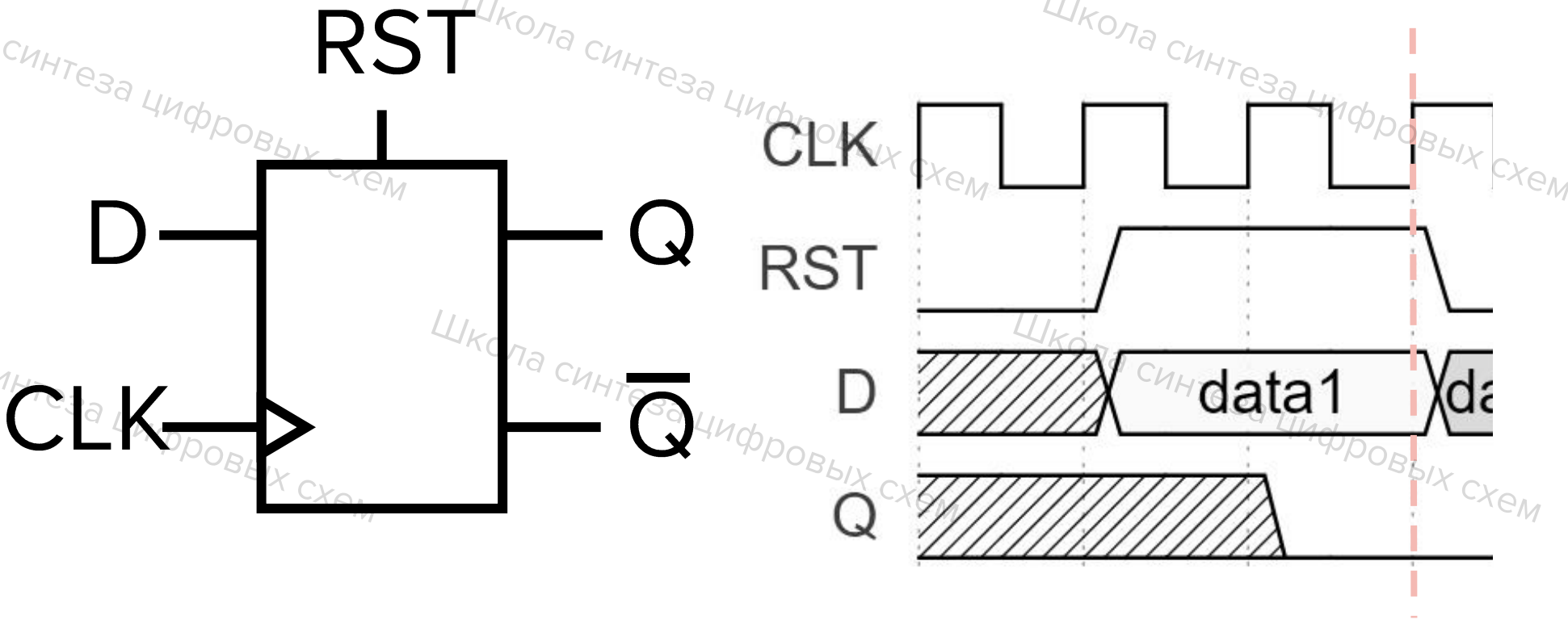
ШКОЛА СИНТЕЗА ЦИФРОВЫХ СХЕМ





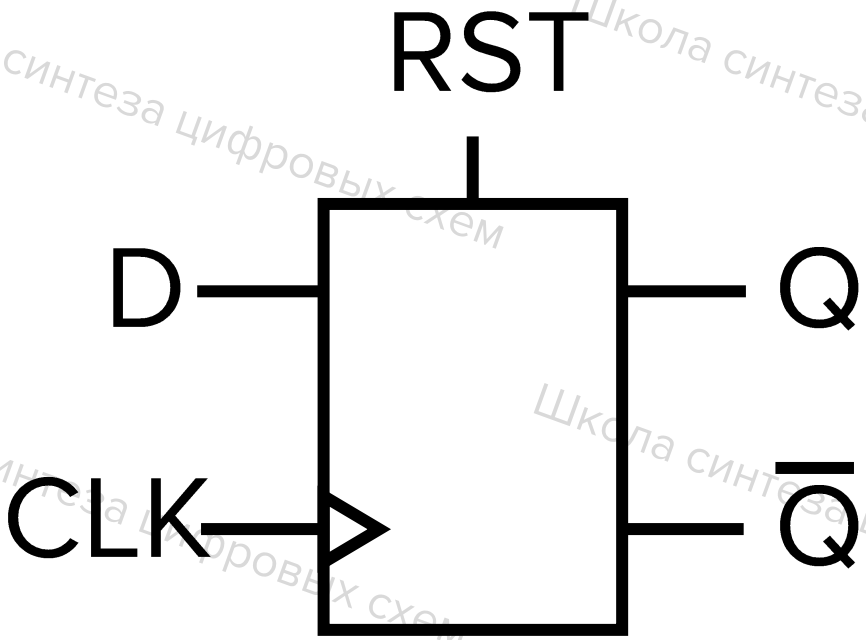
D-ТРИГГЕР. СИГНАЛ СБРОСА

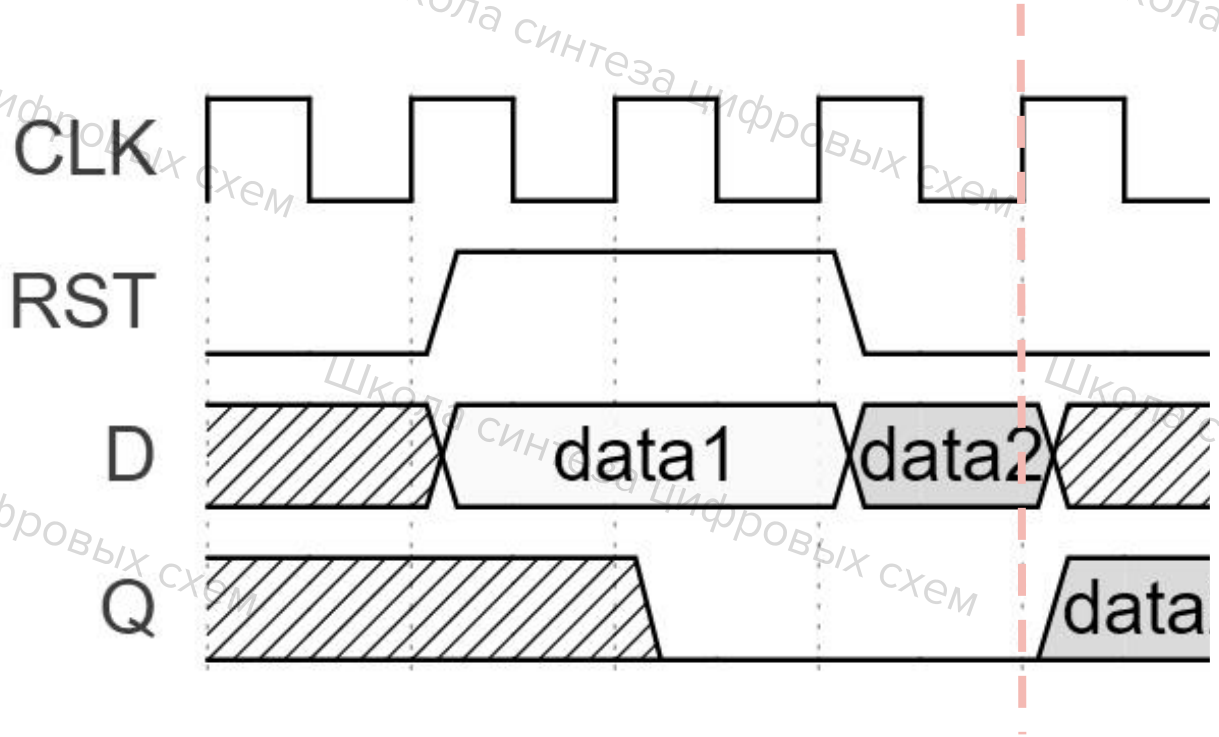
ШКОЛА СИНТЕЗА ЦИФРОВЫХ СХЕМ



DD-ТРИГГЕР. СИГНАЛ СБРОСА

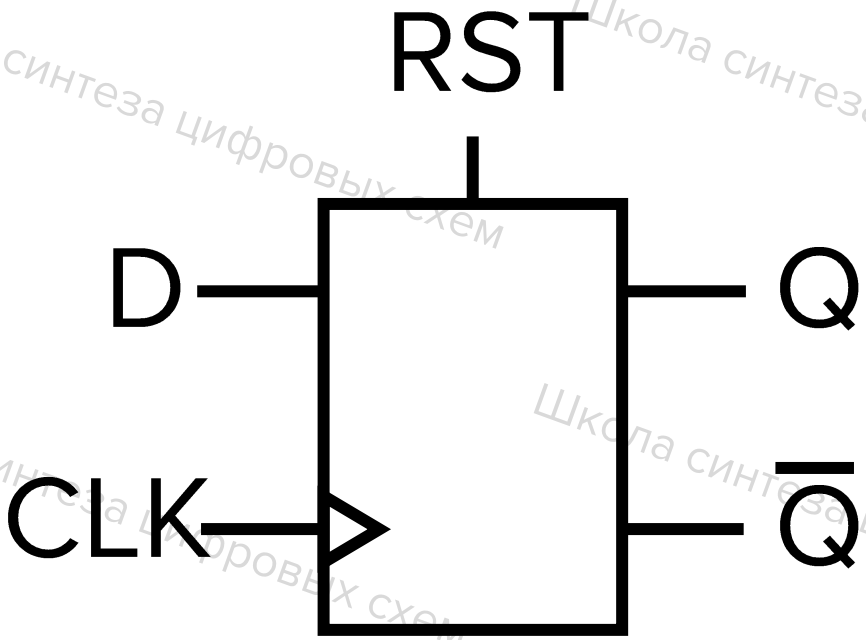
ШКОЛА СИНТЕЗА ЦИФРОВЫХ СХЕМ

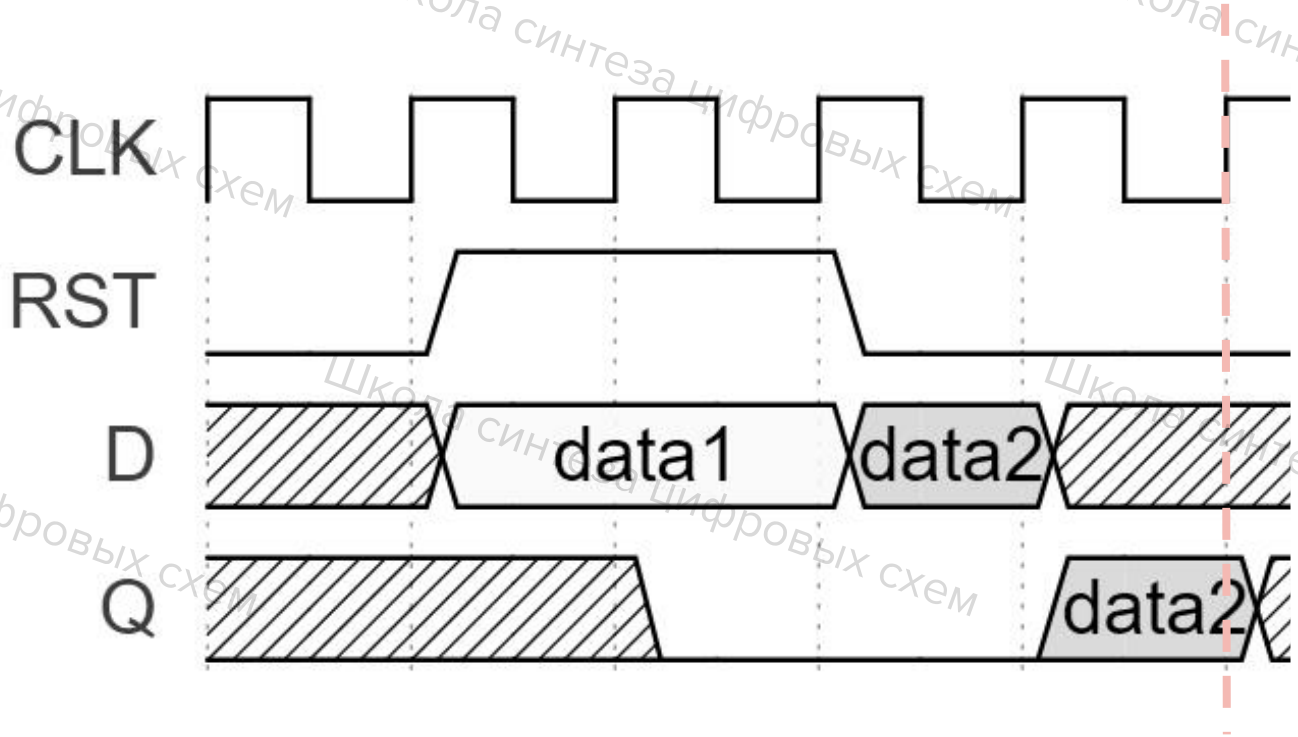




D-ТРИГГЕР. СИГНАЛ СБРОСА

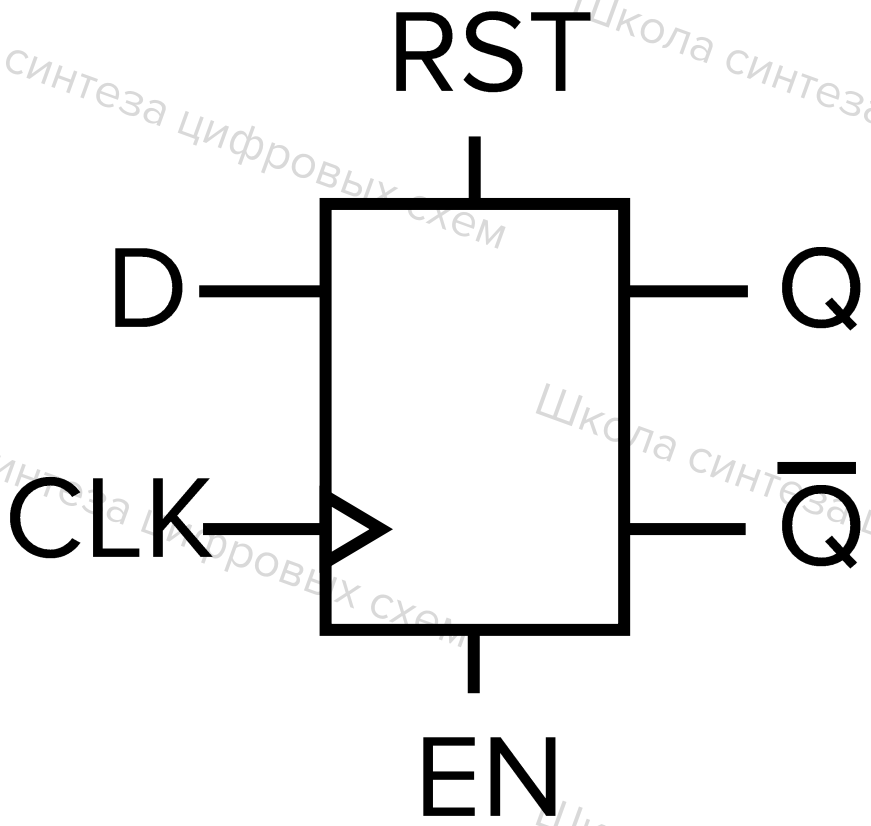
ШКОЛА СИНТЕЗА ЦИФРОВЫХ СХЕМ

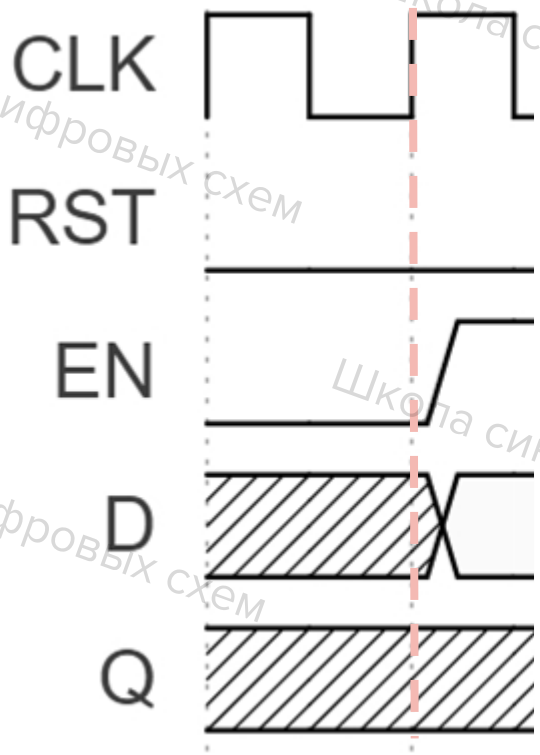




D-ТРИГГЕР. СИГНАЛ РАЗРЕШЕНИЯ

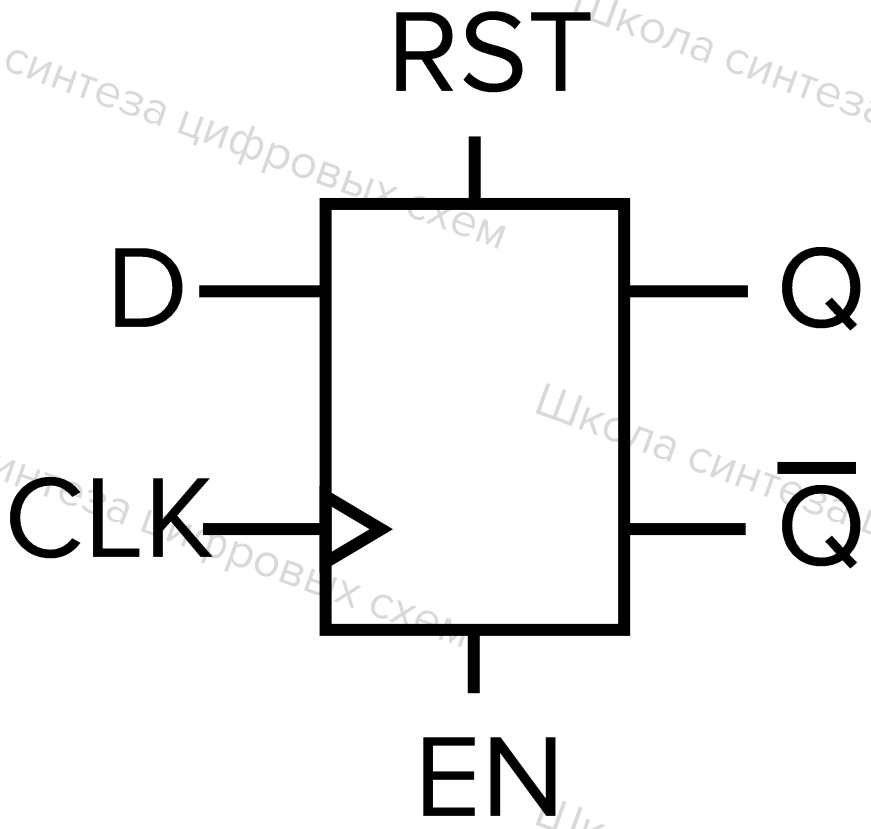
ШКОЛА СИНТЕЗА ЦИФРОВЫХ СХЕМ

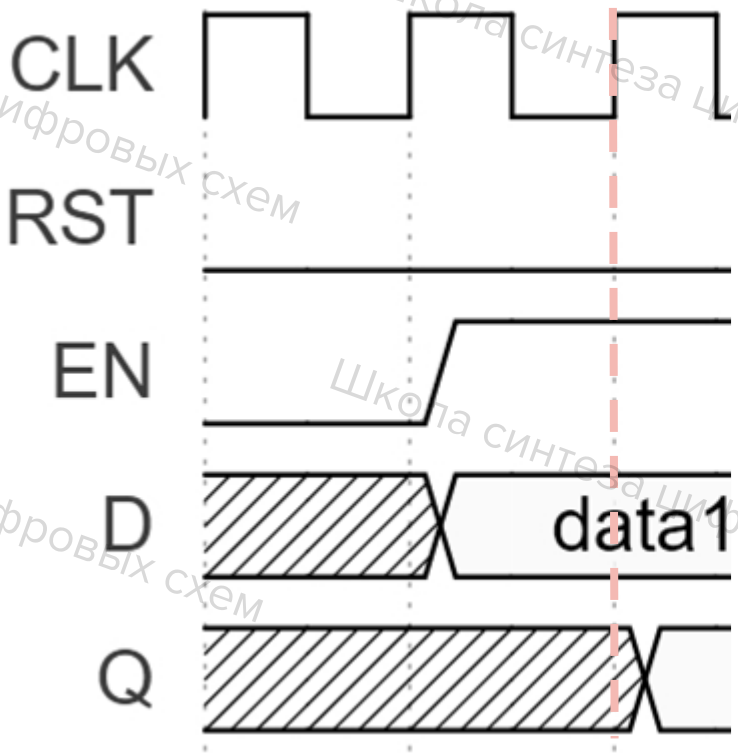




D-ТРИГГЕР. СИГНАЛ РАЗРЕШЕНИЯ

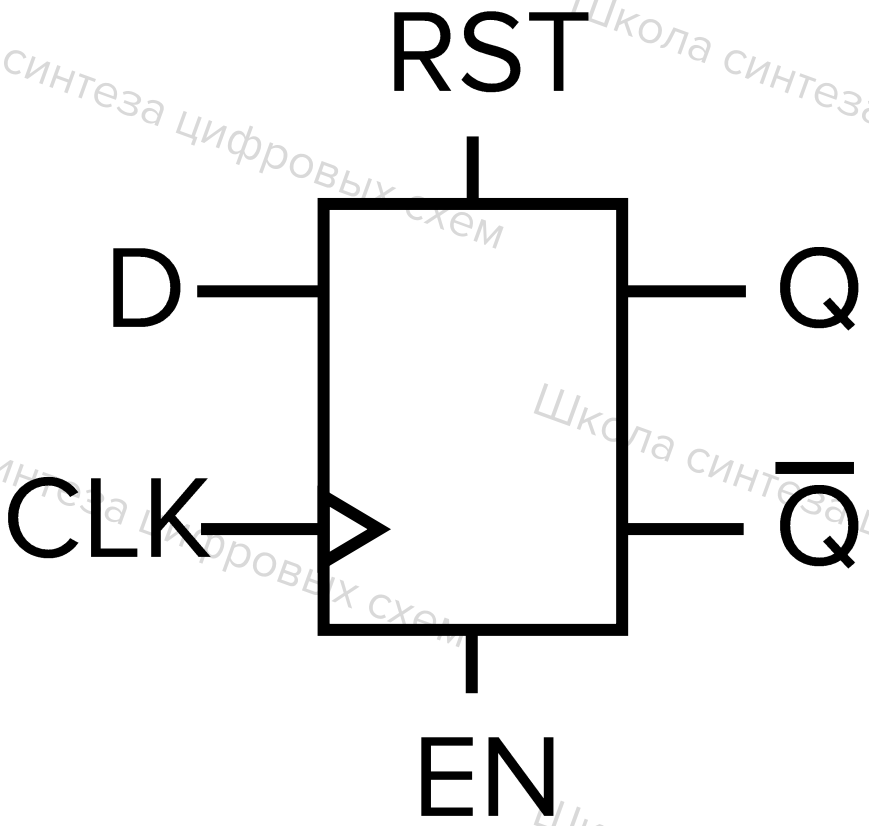
ШКОЛА СИНТЕЗА ЦИФРОВЫХ СХЕМ

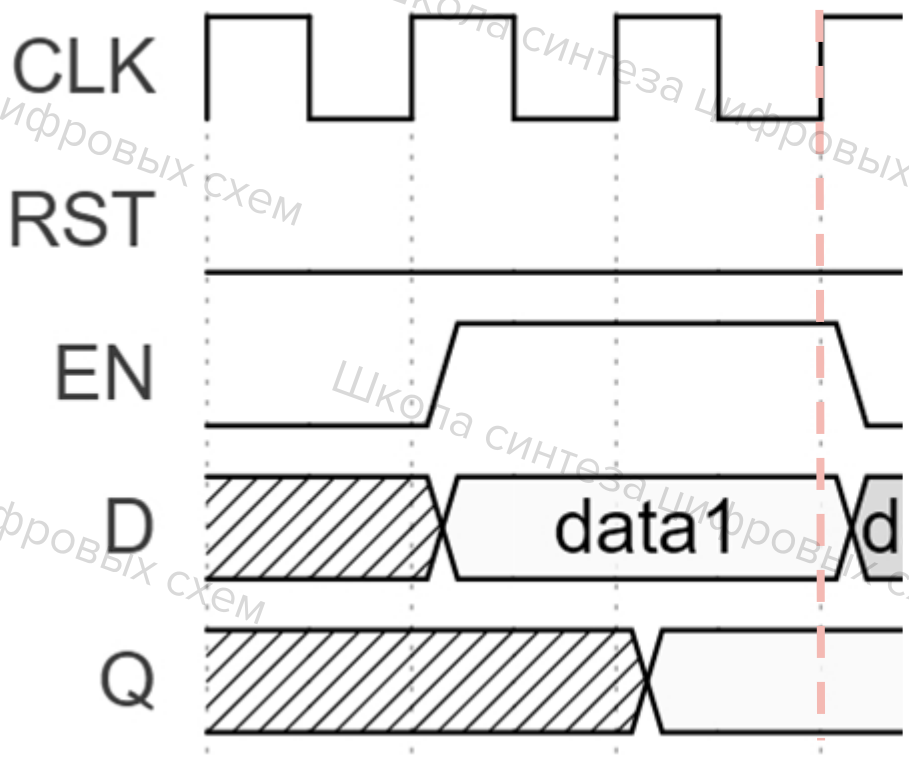




D-ТРИГГЕР. СИГНАЛ РАЗРЕШЕНИЯ

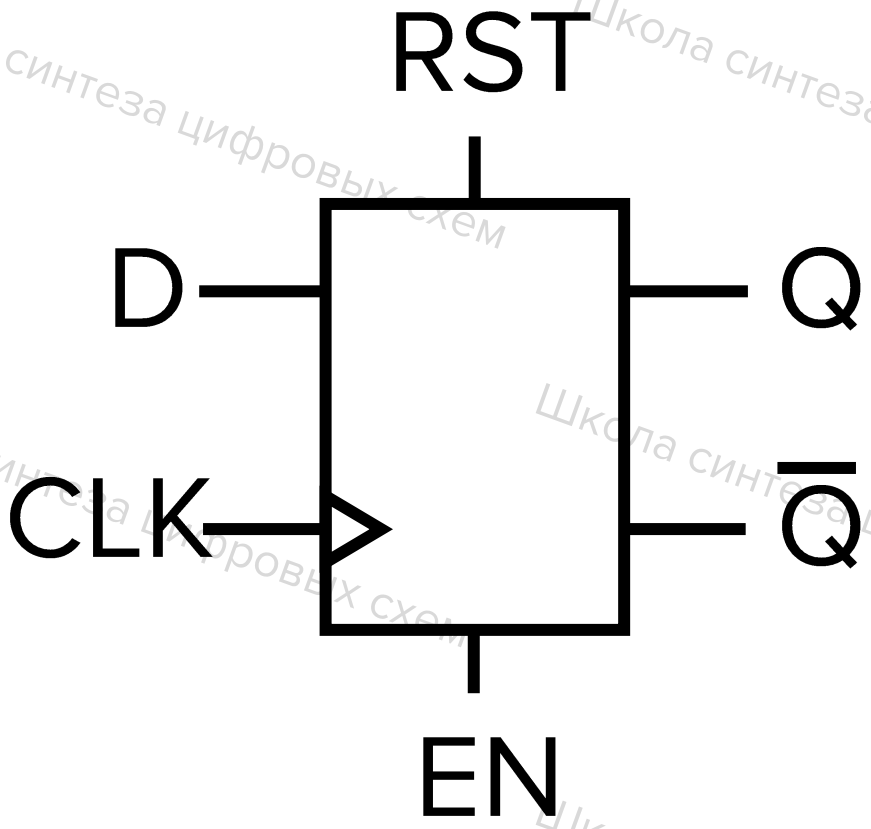
ШКОЛА СИНТЕЗА ЦИФРОВЫХ СХЕМ

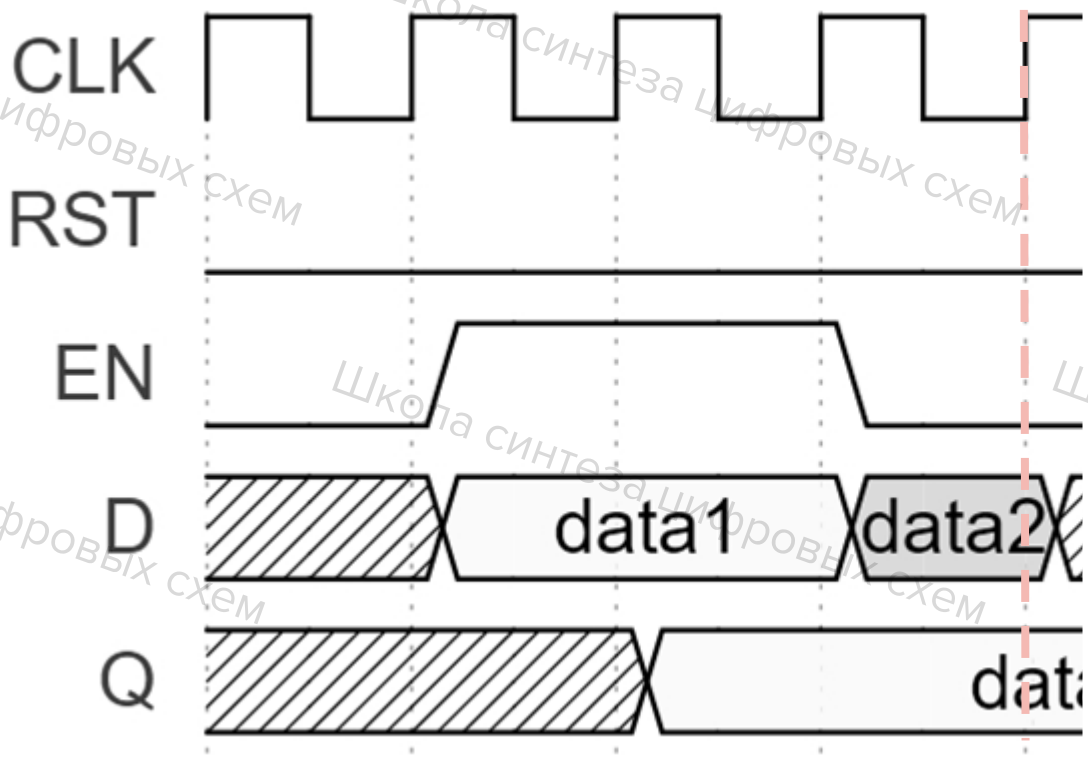




D-ТРИГГЕР. СИГНАЛ РАЗРЕШЕНИЯ

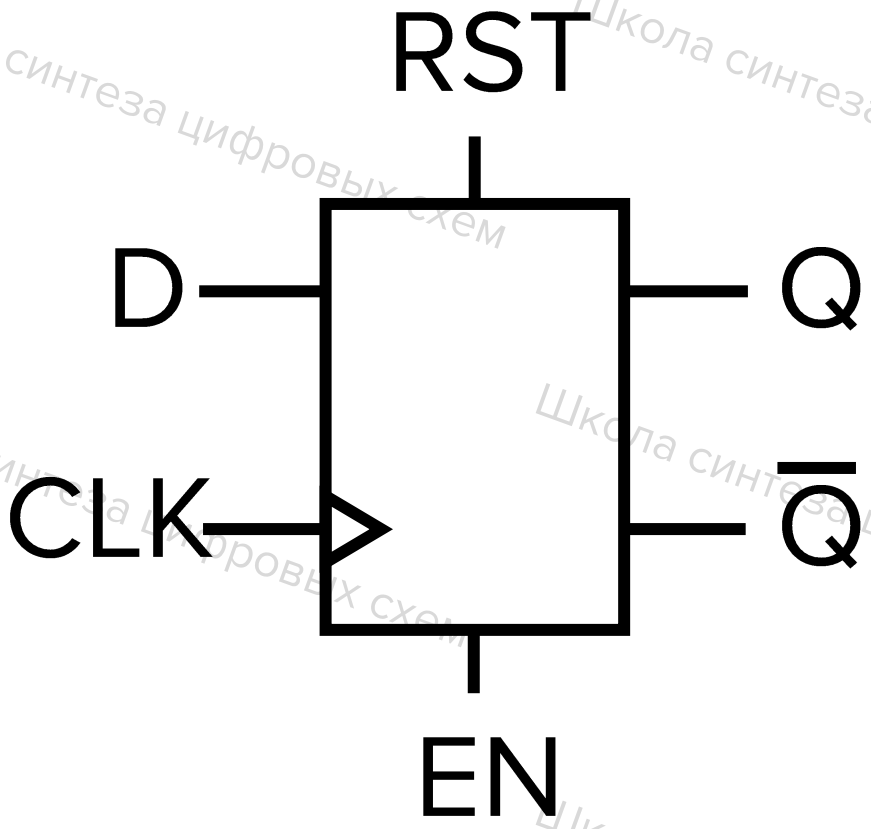
ШКОЛА СИНТЕЗА ЦИФРОВЫХ СХЕМ





D-ТРИГГЕР. СИНХРОННЫЙ СБРОС

ШКОЛА СИНТЕЗА ЦИФРОВЫХ СХЕМ

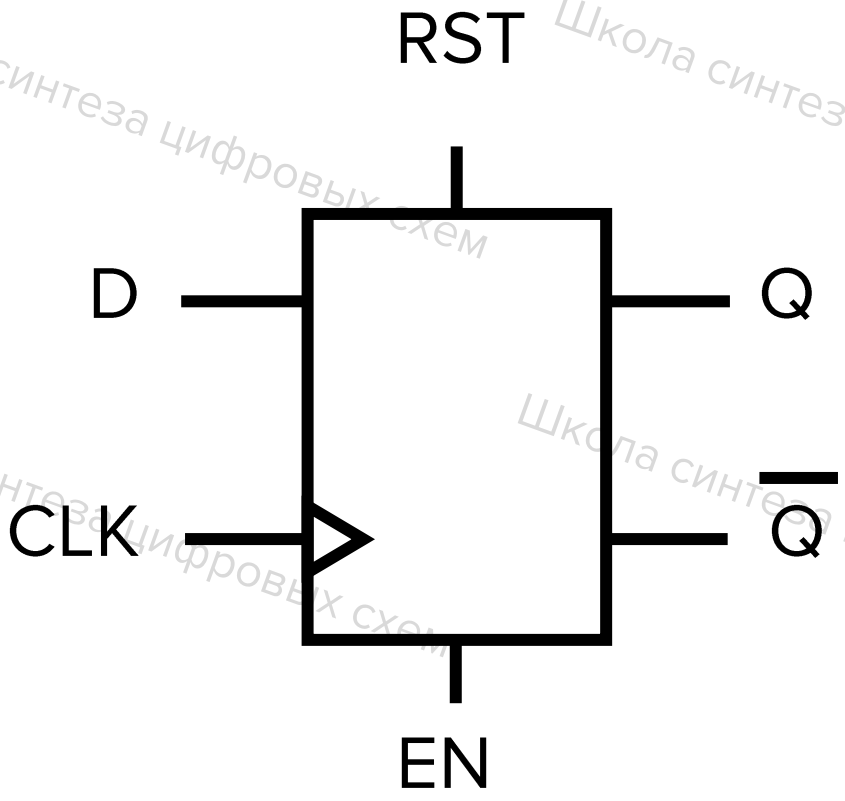



```
module my_reg
(
  input clk_i,
  input rst_i,
  input d_i,
  output q_o
);

logic out;
always_ff @(posedge clk_i) begin
      if(rst_i) out <= 1'b0; else out <= d_i;
end
assign q_o = out;
endmodule
```

D-ТРИГГЕР. АСИНХРОННЫЙ СБРОС

ШКОЛА СИНТЕЗА ЦИФРОВЫХ СХЕМ




```
module my_reg
(
  input clk_i,
  input rst_i,
  input d_i,
  output q_o
);

logic out;
always_ff @(posedge clk_i or posedge rst_i) begin
      if(rst_i) out <= 1'b0; else out <= d_i;
end
assign q_o = out;
endmodule
```

Указание условия асинхронного сброса

школа синтеза цифровых схем

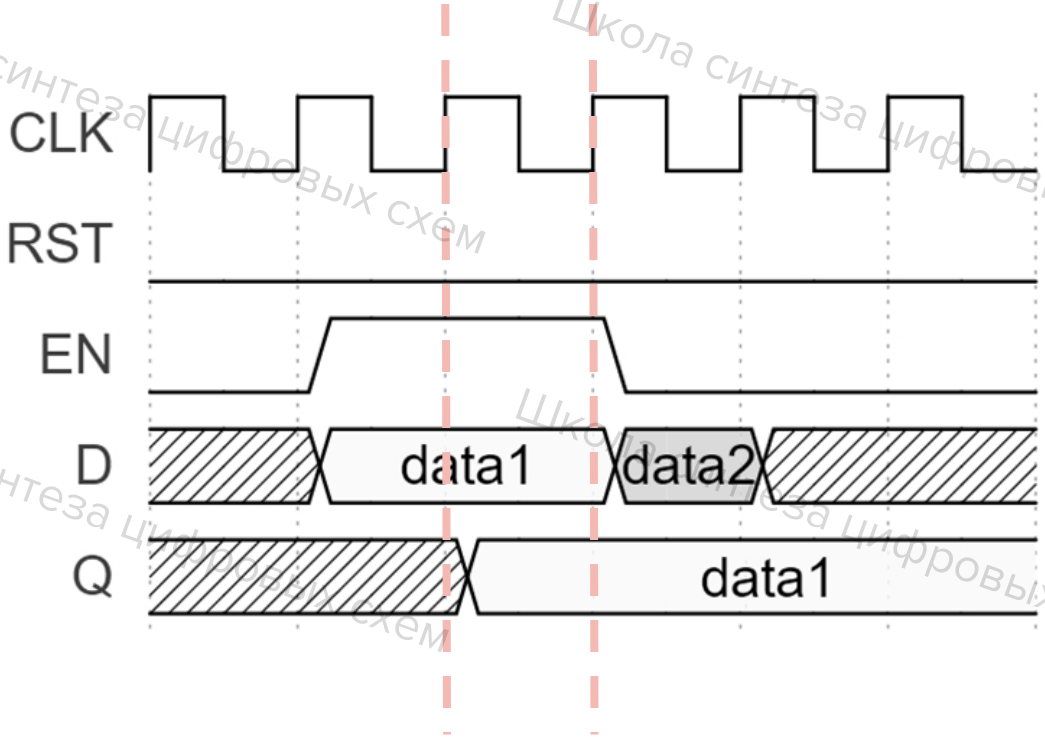
ПРАВИЛА НАИМЕНОВАНИЯ СИГНАЛА СБРОСА

- Сброс может обозначаться в сокращенном или полном виде: reset или rst.
- Префикс в названии сброса указывает на его тип: arst — асинхронный сброс, rst — синхронный.
- Постфикс указывает на активный уровень сброса (то значение, при котором триггер находится в состоянии сброса): arstn — асинхронный сброс с активным уровнем 0, arst — асинхронный сброс с активным уровнем 1.

Школа синтеза цифровых схем

D-ТРИГГЕР. СИГНАЛ РАЗРЕШЕНИЯ

ШКОЛА СИНТЕЗА ЦИФРОВЫХ СХЕМ




```
module my_reg
(
  input clk_i,
  input rst_i,
  input en_i,
  input d_i,
  output q_o
);

logic out;
always_ff @(posedge clk_i or posedge rst_i)
begin
      if(rst_i) out <= 1'b0; else
      if(en_i)  out <= d_i;
end
assign q_o = out;
endmodule
```

Запись в триггер, если EN = 1

школа синтеза цифровых схем

ПРАВИЛА ОПИСАНИЯ ТРИГГЕРОВ ДЛЯ НАЧИНАЮЩИХ

- Для описания триггеров используйте **неблокирующее** присваивание <=

- **Сброс обязателен для всех сигналов контроля.**
  Он необязателен для сигналов данных, если с данными передается специальный бит (valid) который говорит, содержат ли данные корректную информацию.
  При этом сам сигнал valid нужно использовать с сбросом, так как он является контрольным.

- **Нельзя** делать присваивание в один триггер в разных блоках always.

- Описание сброса самое **приоритетное** действие в always блоке.

КОНСТРУКЦИЯ GENERATE

ШКОЛА СИНТЕЗА ЦИФРОВЫХ СХЕМ

```
module add_gen (
        input   [2:0]   A,
        input   [2:0]   B,
        output  [2:0]   S
);

genvar  i;
generate
        for (i=0; i<3; i=i+1) begin : newgen
                adder new (
                        .a(A[i]),
                        .b(B[i]),
                        .s(S[i])
                );
        end
endgenerate

endmodule
```

≡

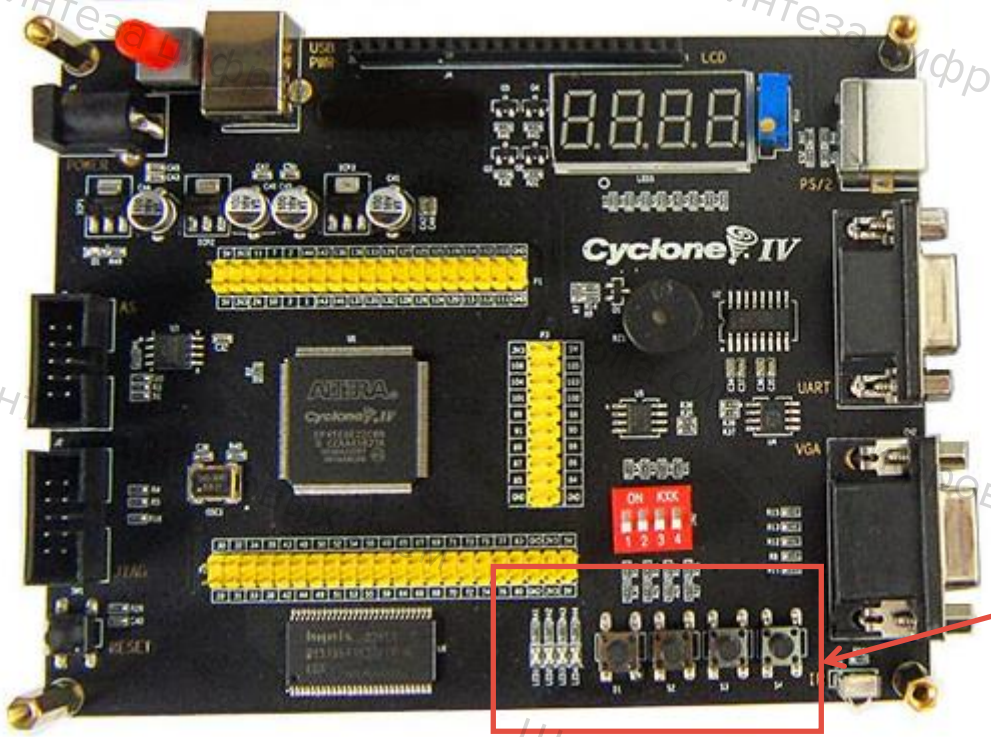
```
adder new0 (
        .a(A[0]),
        .b(B[0]),
        .s(S[0])
);

adder new1 (
        .a(A[1]),
        .b(B[1]),
        .s(S[1])
);

adder new2 (
        .a(A[2]),
        .b(B[2]),
        .s(S[2])
);
```

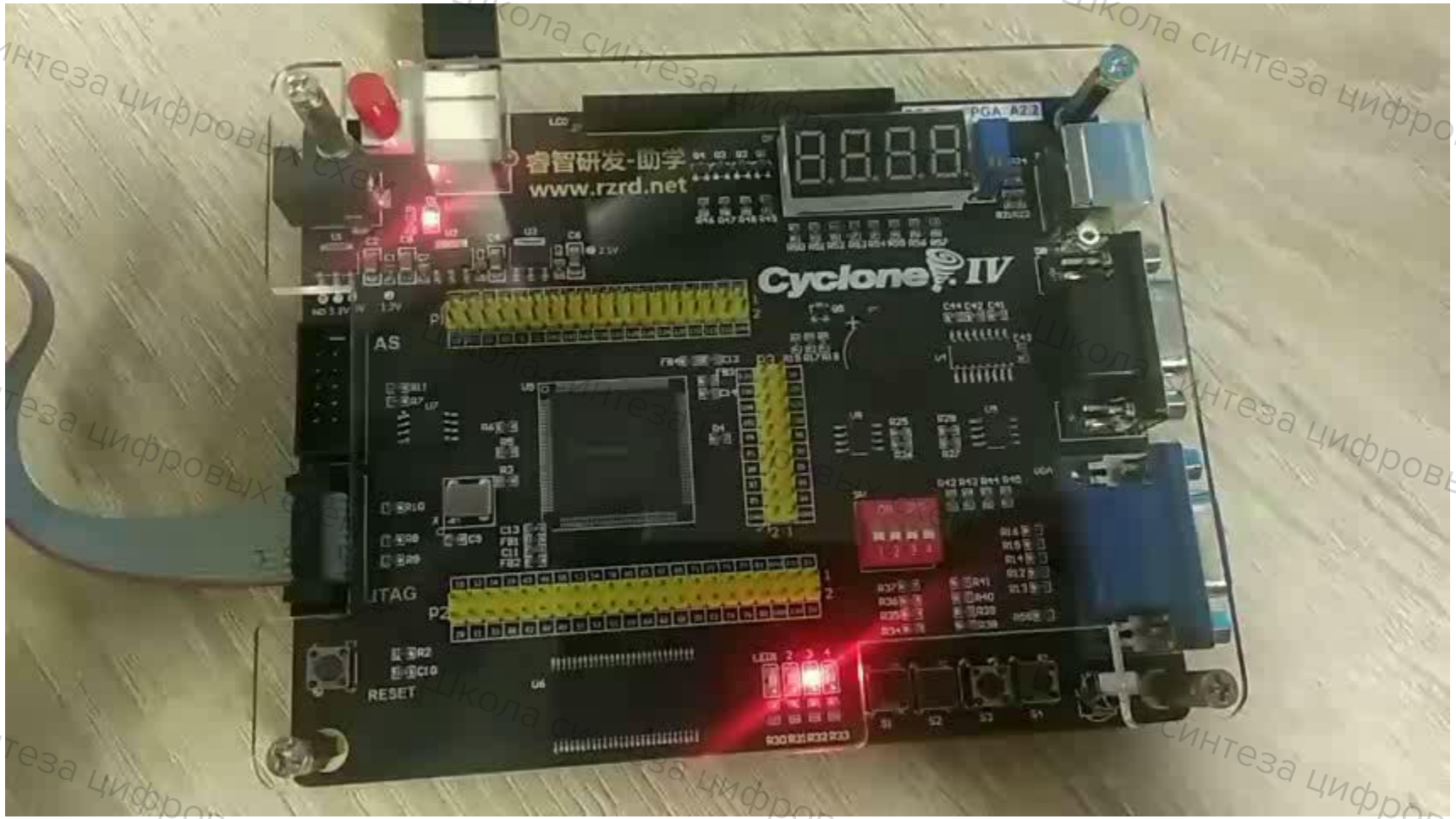
i==0

УПРАЖНЕНИЕ СО СЧЕТЧИКОМ 1_06_BINARY_COUNTER



Вывод значений счетчика на светодиоды.
Использование кнопок для изменения направления счета

УПРАЖНЕНИЕ СО СЧЕТЧИКОМ 1_06_BINARY_COUNTER



Школа синтеза цифровых схем

УПРАЖНЕНИЕ СО СЧЕТЧИКОМ 1_06_BINARY_COUNTER

```
// Exercise 1: Free running counter.
// How do you change the speed of LED blinking?
// Try different bit slices to display.

localparam w_cnt = $clog2 (clk_mhz * 1000 * 1000);

logic [w_cnt - 1:0] cnt;
```



УПРАЖНЕНИЕ СО СЧЕТЧИКОМ 1_06_BINARY_COUNTER

```
// Exercise 1: Free running counter.
// How do you change the speed of LED blinking?
// Try different bit slices to display.

localparam w_cnt = $clog2 (clk_mhz * 1000 * 1000);

logic [w_cnt - 1:0] cnt;
```

Объявление многоразрядного счетчика cnt

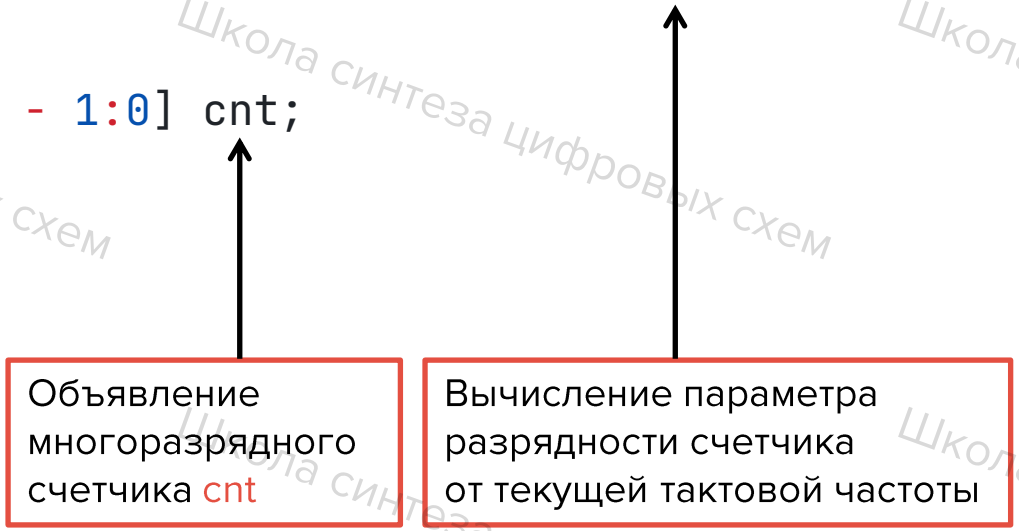
Вычисление параметра разрядности счетчика от текущей тактовой частоты

УПРАЖНЕНИЕ СО СЧЕТЧИКОМ 1_06_BINARY_COUNTER

```
// Exercise 1: Free running counter.
// How do you change the speed of LED blinking?
// Try different bit slices to display.

localparam w_cnt = $clog2 (clk_mhz * 1000 * 1000);

logic [w_cnt - 1:0] cnt;
```



- clog2 — встроенная системная функция вычисления логарифма по основанию 2.
- $clog2 – встроенная системная функция вычисления логарифма по основанию 2, с округлением вверх. Результат $clog2 равен количеству бит, в которые можно вместить числа от 0 до N-1, где N - аргумент $clog2
- можно использовать в синтезируемом коде только тогда, когда ее параметр — это константное выражение, которое состоит из числовых констрант, а также parameter, localparam и `define выражений, которые сводятся к константам.
- Все системные функции в SystemVerilog начинаются с $

УПРАЖНЕНИЕ СО СЧЕТЧИКОМ 1_06_BINARY_COUNTER

школа синтеза цифровых схем

```
always_ff @ (posedge clk or posedge rst)
    if (rst)
        cnt <= '0;
    else
        cnt <= cnt + 1'd1;

    assign led = cnt [$left (cnt) -: w_led];
```

Асинхронный сброс

Школа синтеза цифровых схем

УПРАЖНЕНИЕ СО СЧЕТЧИКОМ 1_06_BINARY_COUNTER

```
always_ff @ (posedge clk or posedge rst)
    if (rst)
        cnt <= '0;
    else
        cnt <= cnt + 1'd1;

    assign led = cnt [$left (cnt) -: w_led];
```

Активный уровень сброса 1

УПРАЖНЕНИЕ СО СЧЕТЧИКОМ 1_06_BINARY_COUNTER

```
always_ff @ (posedge clk or posedge rst)
    if (rst)
        cnt <= '0;
    else
        cnt <= cnt + 1'd1;

    assign led = cnt [$left (cnt) -: w_led];
```

Начальное значение счетчика 0

УПРАЖНЕНИЕ СО СЧЕТЧИКОМ 1_06_BINARY_COUNTER

```
always_ff @ (posedge clk or posedge rst)
    if (rst)
        cnt <= '0;
    else
        cnt <= cnt + 1'd1;

    assign led = cnt [$left (cnt) -: w_led];
```

Инкрементация счетчика каждый такт на 1,
если нет сигнала активного сброса.

При достижении максимального значения переполняется и переходит в значение 0.

Если счетчик 4 разрядный и равен 4'b1111,

то следующее значение 0

УПРАЖНЕНИЕ СО СЧЕТЧИКОМ 1_06_BINARY_COUNTER

```
always_ff @ (posedge clk or posedge rst)
    if (rst)
        cnt <= '0;
    else
        cnt <= cnt + 1'd1;

    assign led = cnt [$left (cnt) -: w_led];
```

На светодиоды выводится диапазон разрядов счетчика. Чем больше индекс разряда, тем с меньшей частотой мигает светодиод.

УПРАЖНЕНИЕ СО СЧЕТЧИКОМ 1_06_BINARY_COUNTER

```
always_ff @ (posedge clk or posedge rst)
    if (rst)
        cnt <= '0;
    else
        cnt <= cnt + 1'd1;

assign led = cnt [$left (cnt) -: w_led];
```

На светодиоды выводится диапазон разрядов счетчика. Чем больше индекс разряда, тем с меньшей частотой мигает светодиод.

$left(cnt) возвращает номер старшего индекса в cnt. Если cnt 8-разрядный, то вернется 7.
Если $left (cnt) = 7 а w_led = 3
то [$left (cnt) -: w_led] эквивалентно [7:5]

ШКОЛА СИНТЕЗА ЦИФРОВЫХ СХЕМ

УПРАЖНЕНИЕ СО СЧЕТЧИКОМ 1_06_BINARY_COUNTER

```
logic [31:0] a_vect;
logic [0:31] b_vect;
logic [63:0] dword;
integer sel;

a_vect[0  +:8]      // ==a_vect[7:0]
a_vect[15 -:8]      // ==a_vect[15:8]

b_vect[0  +:8]       // ==b_vect[0:7]
b_vect[15 -:8]       // ==b_vect[8:15]

dword[8*sel +:8]     // variable part-select with
fixed width
```

w[x+:y] эквивалентно w[x :(x+y – 1)]
w[x-:y] эквивалентно w[x :(x-y + 1)]

$left(cnt) возвращает номер старшего Индекса в cnt. Если cnt 8-разрядный, То вернется 7.
Если $left(cnt) = 7, а w_led = 3, то [$left(cnt) -: w_led] эквивалентно [7:5]

школа синтеза цифровых схем

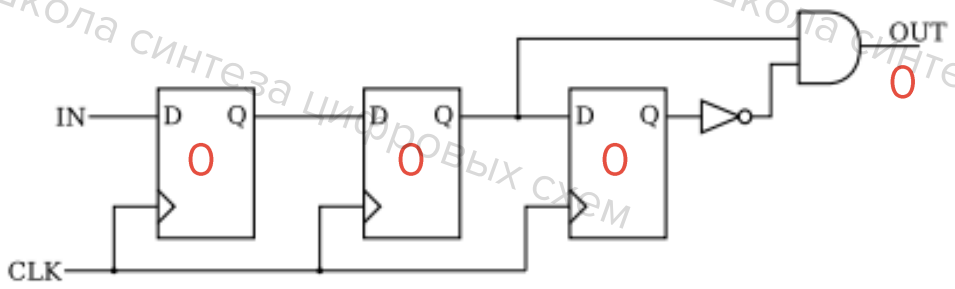
УПРАЖНЕНИЕ СО СЧЕТЧИКОМ 1_06_BINARY_COUNTER

```
wire any_key = | key;

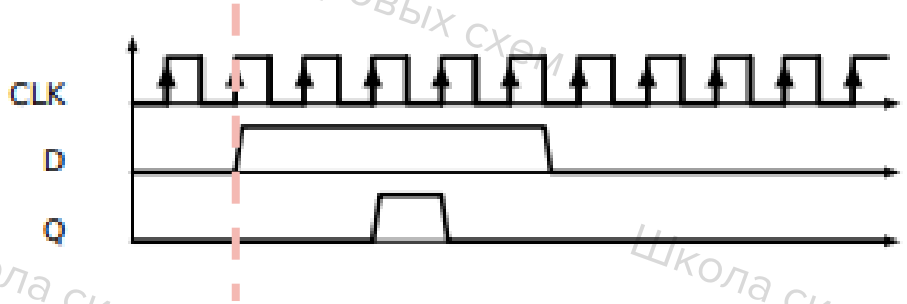
logic any_key_r;

always_ff @ (posedge clk or posedge rst)
    if (rst)
        any_key_r <= '0;
    else
        any_key_r <= any_key;

wire any_key_pressed = ~ any_key & any_key_r;
```



Пример подобной схемы с тремя регистрами.
И временная диаграмма отражающая работу



школа синтеза цифровых схем

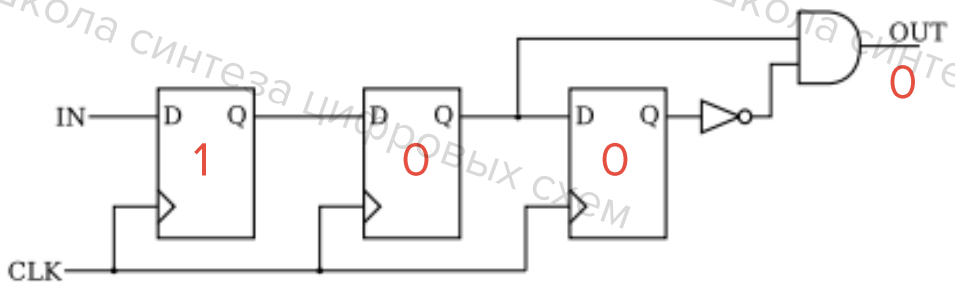
УПРАЖНЕНИЕ СО СЧЕТЧИКОМ 1_06_BINARY_COUNTER

```
wire any_key = | key;

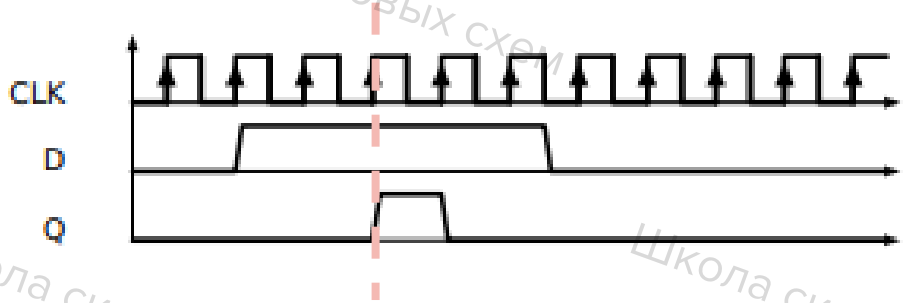
logic any_key_r;

always_ff @ (posedge clk or posedge rst)
    if (rst)
        any_key_r <= '0;
    else
        any_key_r <= any_key;

wire any_key_pressed = ~ any_key & any_key_r;
```



Пример подобной схемы с тремя регистрами.
И временная диаграмма отражающая работу



школа синтеза цифровых схем

УПРАЖНЕНИЕ СО СЧЕТЧИКОМ 1_06_BINARY_COUNTER

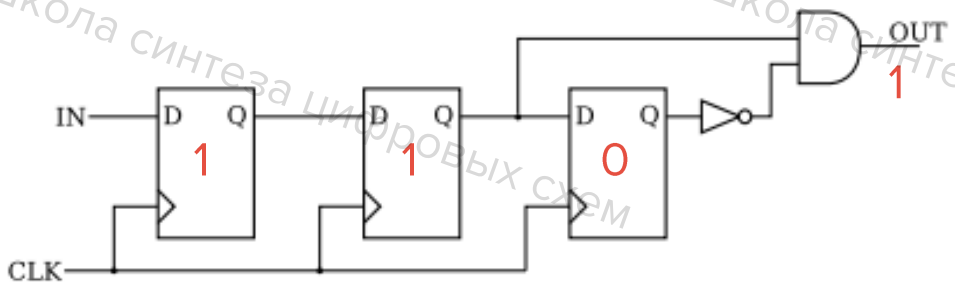
```
wire any_key = | key;

logic any_key_r;

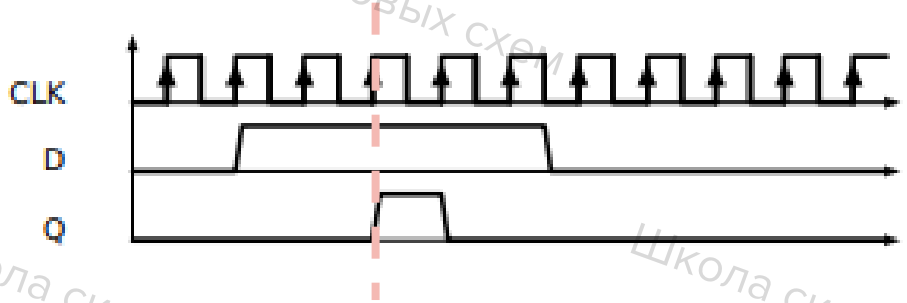
always_ff @ (posedge clk or posedge rst)
    if (rst)
        any_key_r <= '0;
    else
        any_key_r <= any_key;

wire any_key_pressed = ~ any_key & any_key_r;
```

Превращение нажатия на любую кнопку в однотактный строб для однократной обработки нажатия на кнопки



Пример подобной схемы с тремя регистрами.
И временная диаграмма отражающая работу



УПРАЖНЕНИЕ СО СЧЕТЧИКОМ 1_06_BINARY_COUNTER

```
logic [w_led - 1:0] cnt;

always_ff @ (posedge clk or posedge rst)
    if (rst)
        cnt <= '0;
    else if (any_key_pressed)
        cnt <= cnt + 1'd1;

assign led = w_led' (cnt);
```

Счетчик инкрементируется на 1,
если в момент положительного фронта
clk any_key_pressed равен 1
В итоге каждое новое нажатие на кнопку
увеличивает счетчик на 1

УПРАЖНЕНИЕ СО СЧЕТЧИКОМ 1_06_BINARY_COUNTER

школа синтеза цифровых схем

Упражнение 1

Свободно инкрементирующийся счетчик.

- Измените скорость мигания светодиода. Попробуйте разные диапазоны битов для отображения.

Закомментируйте решение первого упражнения, перед решением второго

Упражнение 2

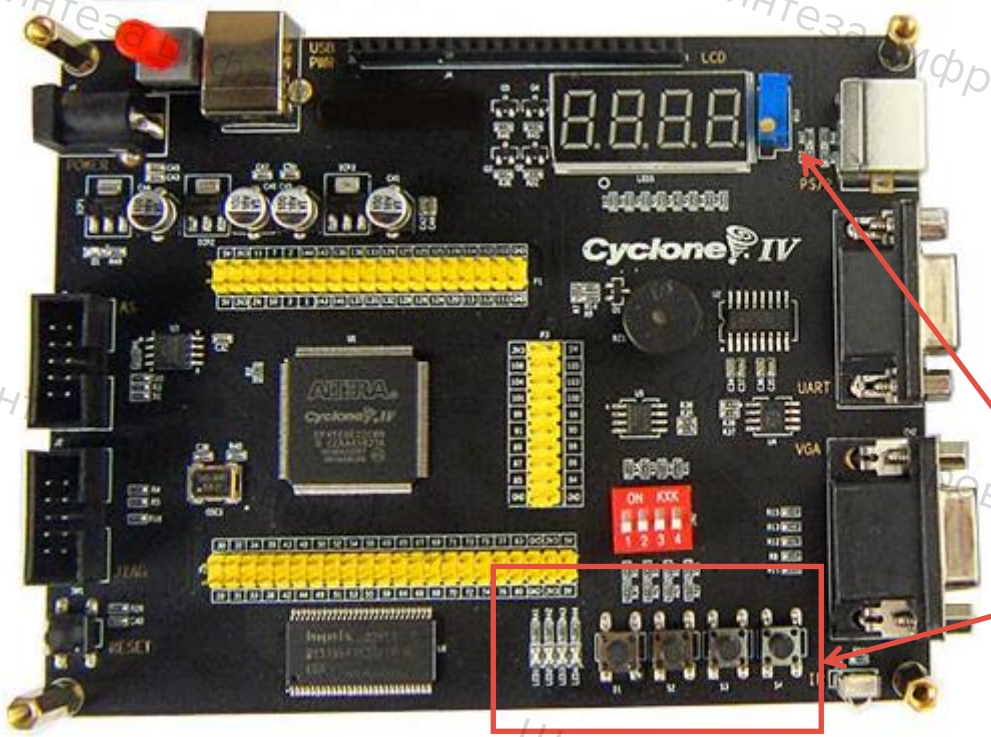
Счетчик, управляемый кнопкой.

Раскомментируйте и синтезируйте код.

Нажмите кнопку, чтобы увидеть увеличение счетчика.

- Измените дизайн:
  1. Одна кнопка используется для увеличения, другая для уменьшения значения счетчика.
  2. Сделайте два счетчика, управляемые разными кнопками и отобразите их в разных группах светодиодов.

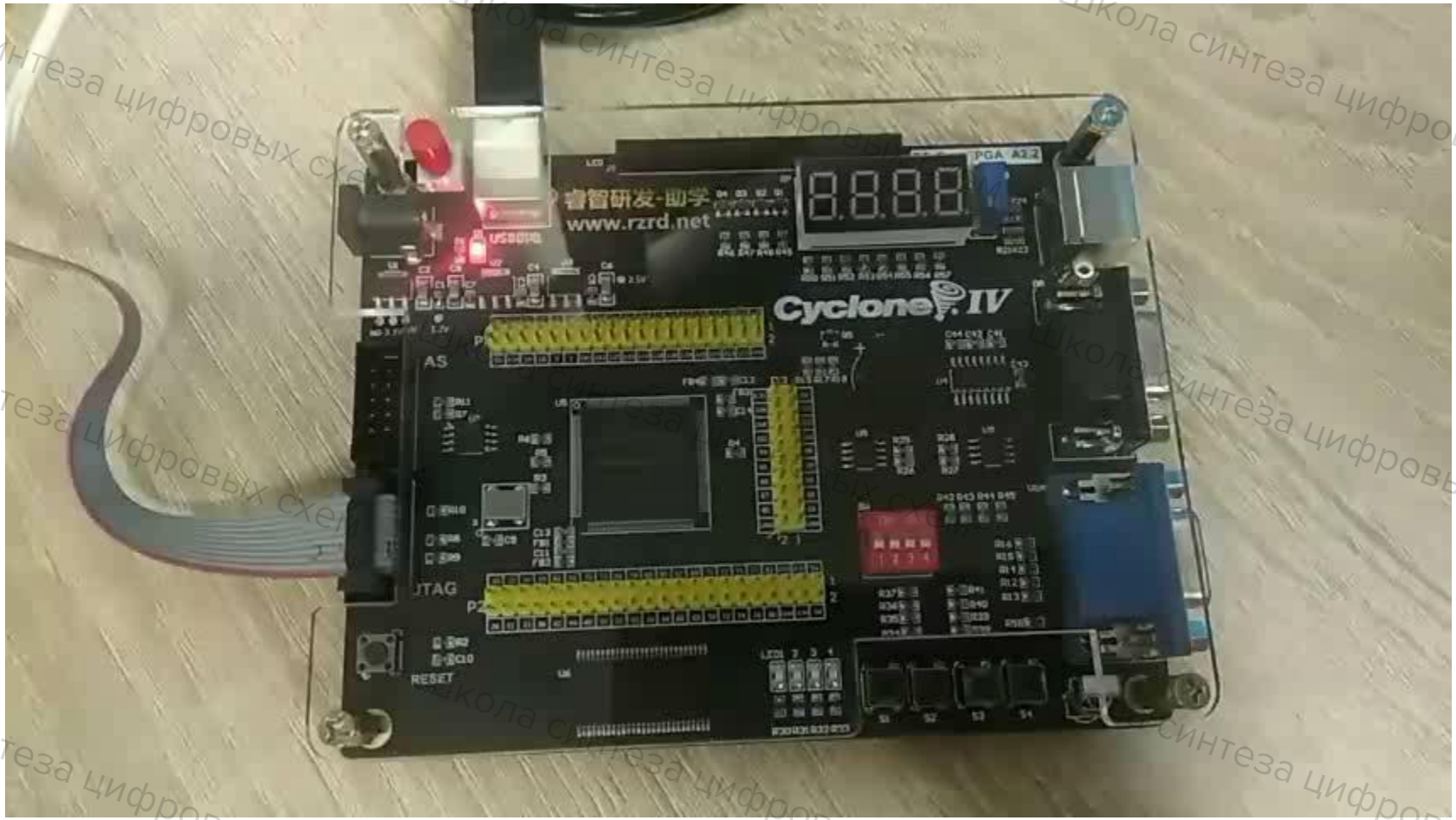
УПРАЖНЕНИЕ СО СДВИГОВЫМ РЕГИСТРОМ 1_07_SHIFT_REGISTER



Вывод значений счетчика на светодиоды.
Использование кнопок для изменения направления счета

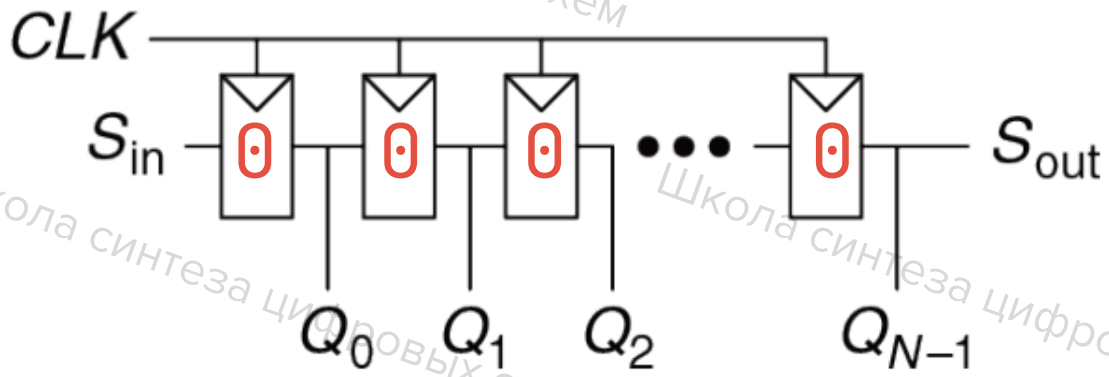
школа синтеза цифровых схем

УПРАЖНЕНИЕ СО СДВИГОВЫМ РЕГИСТРОМ 1_07_SHIFT_REGISTER



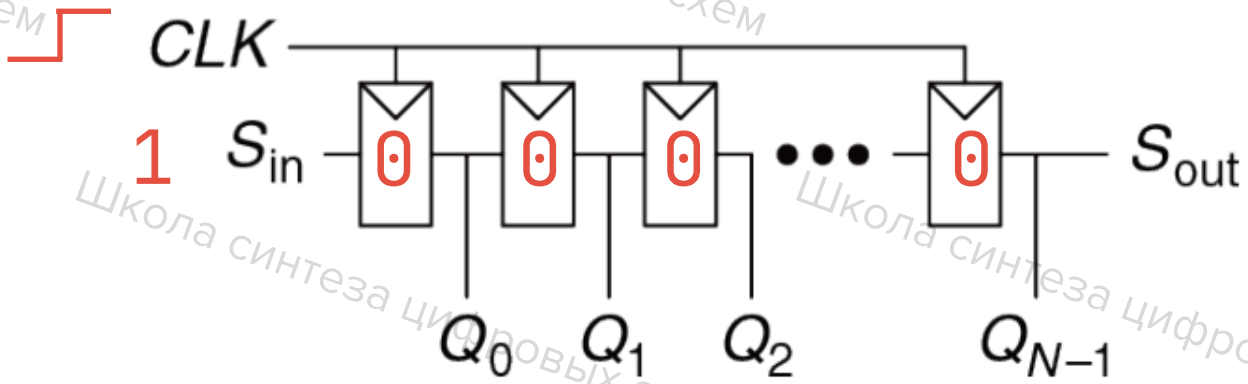
СДВИГОВЫЙ РЕГИСТР

школа синтеза цифровых схем



СДВИГОВЫЙ РЕГИСТР

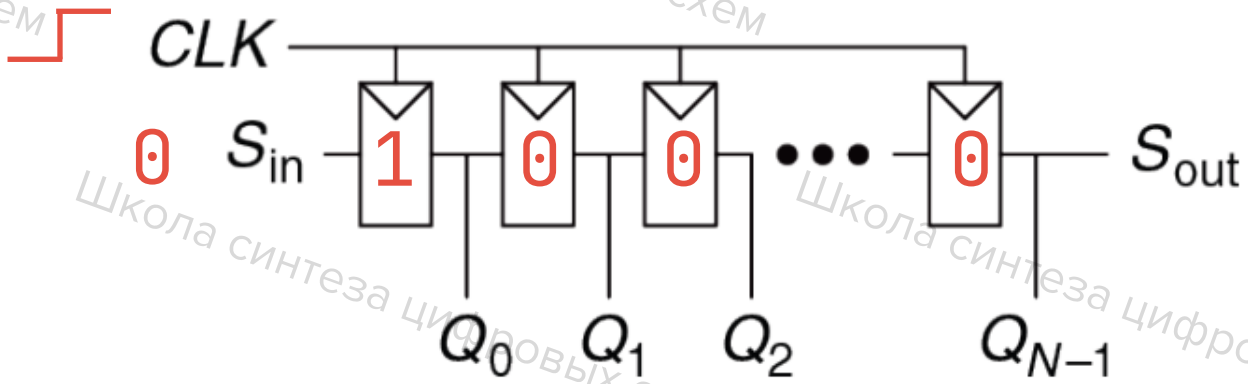
школа синтеза цифровых схем



Школа синтеза цифровых схем

СДВИГОВЫЙ РЕГИСТР

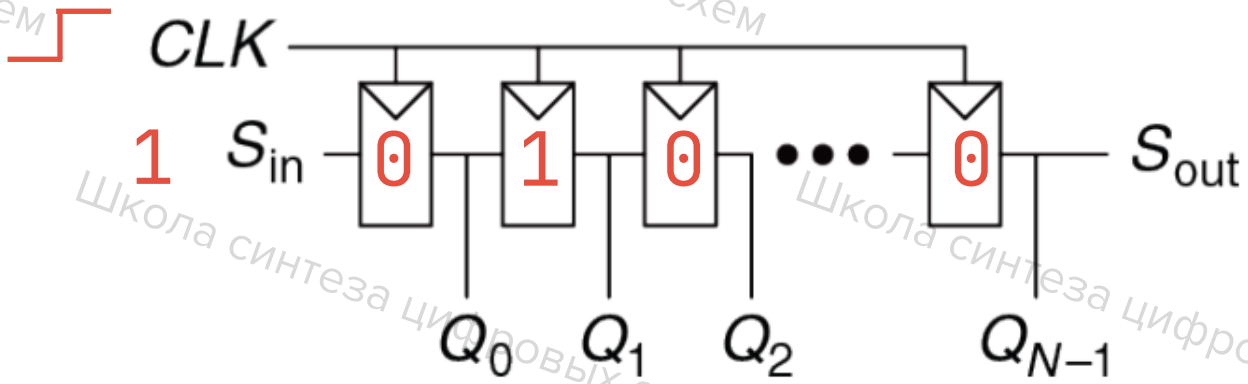
школа синтеза цифровых схем



Школа синтеза цифровых схем

СДВИГОВЫЙ РЕГИСТР

школа синтеза цифровых схем



Школа синтеза цифровых схем

УПРАЖНЕНИЕ СО СДВИГОВЫМ РЕГИСТРОМ 1_07_SHIFT_REGISTER

```
logic [31:0] cnt;

always_ff @ (posedge clk or posedge rst)
    if (rst)
        cnt <= '0;
    else
        cnt <= cnt + 1'd1;

wire enable = (cnt [22:0] == '0);
```

Генерация сигнала enable который равен 1 один раз в 2**23 тактов.
Изменив присваиваемый диапазон, можно изменить частоту генерации сигнала enable.
Данный сигнал необходим для удобной визуализации работы сдвигового регистра в примере

УПРАЖНЕНИЕ СО СДВИГОВЫМ РЕГИСТРОМ 1_07_SHIFT_REGISTER

школа синтеза цифровых схем

```
wire button_on = | key;

logic [w_led - 1:0] shift_reg;

always_ff @ (posedge clk or posedge rst)
    if (rst)
        shift_reg <= '1;
    else if (enable)
        shift_reg <= { button_on, shift_reg [w_led - 1:1] };

assign led = shift_reg;
```

Нажатие на любую кнопку дает button_on =1

Школа синтеза цифровых схем

УПРАЖНЕНИЕ СО СДВИГОВЫМ РЕГИСТРОМ 1_07_SHIFT_REGISTER

```
wire button_on = | key;

logic [w_led - 1:0] shift_reg;

always_ff @ (posedge clk or posedge rst)
    if (rst)
        shift_reg <= '1;
    else if (enable)
        shift_reg <= { button_on, shift_reg [w_led - 1:1] };

assign led = shift_reg;
```

Объявление сдвигового регистра разрядности той же, что и количество led на вашей плате. Если мы хотим, чтобы по сдвиговому регистру двигали не однобитные значения то его надо объявить двумерным массивом регистров

УПРАЖНЕНИЕ СО СДВИГОВЫМ РЕГИСТРОМ 1_07_SHIFT_REGISTER

ШКОЛА СИНТЕЗА ЦИФРОВЫХ СХЕМ

```
wire button_on = | key;

logic [w_led - 1:0] shift_reg;

always_ff @ (posedge clk or posedge rst)
    if (rst)
        shift_reg <= '1;
    else if (enable)
        shift_reg <= { button_on, shift_reg [w_led - 1:1] };

assign led = shift_reg;
```

Реализация сдвига вправо на 1 разряд на такт с помощью операции конкатенации.
Причем сдвиг будет осуществляться только, когда enable = 1

УПРАЖНЕНИЕ СО СДВИГОВЫМ РЕГИСТРОМ 1_07_SHIFT_REGISTER

```
wire button_on = | key;

logic [w_led - 1:0] shift_reg;

always_ff @ (posedge clk or posedge rst)
    if (rst)
        shift_reg <= '1;
    else if (enable)
        shift_reg <= { button_on, shift_reg [w_led - 1:1] };

assign led = shift_reg;
```

Вывод значения сдвигового регистра на светодиоды.
В итоге на led при нажатии на кнопку будет видно движение огоньков по сдвиговому регистру

УПРАЖНЕНИЕ СО СДВИГОВЫМ РЕГИСТРОМ 1_07_SHIFT_REGISTER

```
wire button_on = | key;

logic [w_led - 1:0] shift_reg;

always_ff @ (posedge clk or posedge rst)
    if (rst)
        shift_reg <= '1;
    else if (enable)
        shift_reg <= { button_on, shift_reg [w_led - 1:1] };

assign led = shift_reg;
```

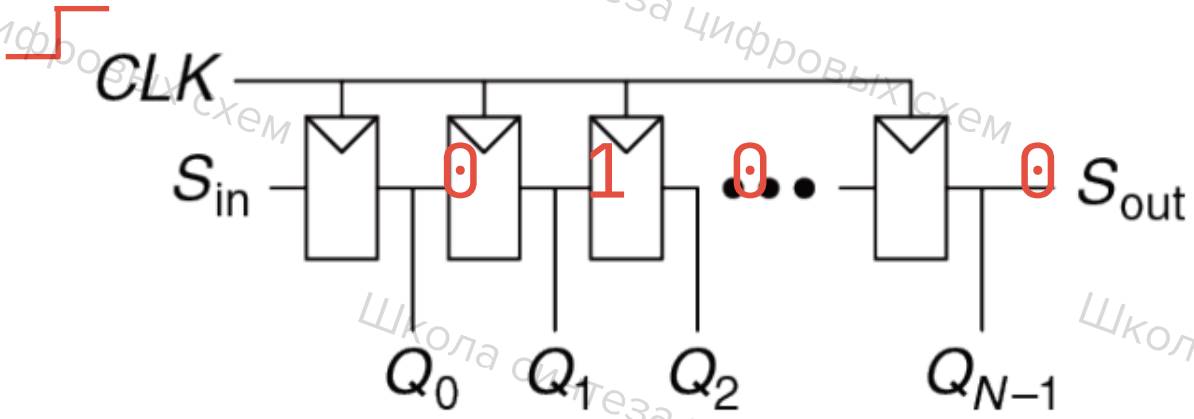
Для реализации циклического сдвига надо каждый такт вдвигать не button_on, а выталкиваемый разряд shift_reg

школа синтеза цифровых схем

УПРАЖНЕНИЕ СО СДВИГОВЫМ РЕГИСТРОМ 1_07_SHIFT_REGISTER

```
always_ff @ (posedge clk or posedge rst)
    if (rst)
        shift_reg <= '1;
    else if (enable)
        shift_reg <= { button_on, shift_reg [w_led - 1:1] };
```

Для реализации циклического сдвига надо каждый такт вдвигать не button_on, а выталкиваемый разряд shift_reg



УПРАЖНЕНИЕ СО СДВИГОВЫМ РЕГИСТРОМ 1_07_SHIFT_REGISTER

ШКОЛА СИНТЕЗА ЦИФРОВЫХ СХЕМ

Упражнение 1

- Заставьте огоньки двигаться в противоположном направлении на светодиодах.

Упражнение 2

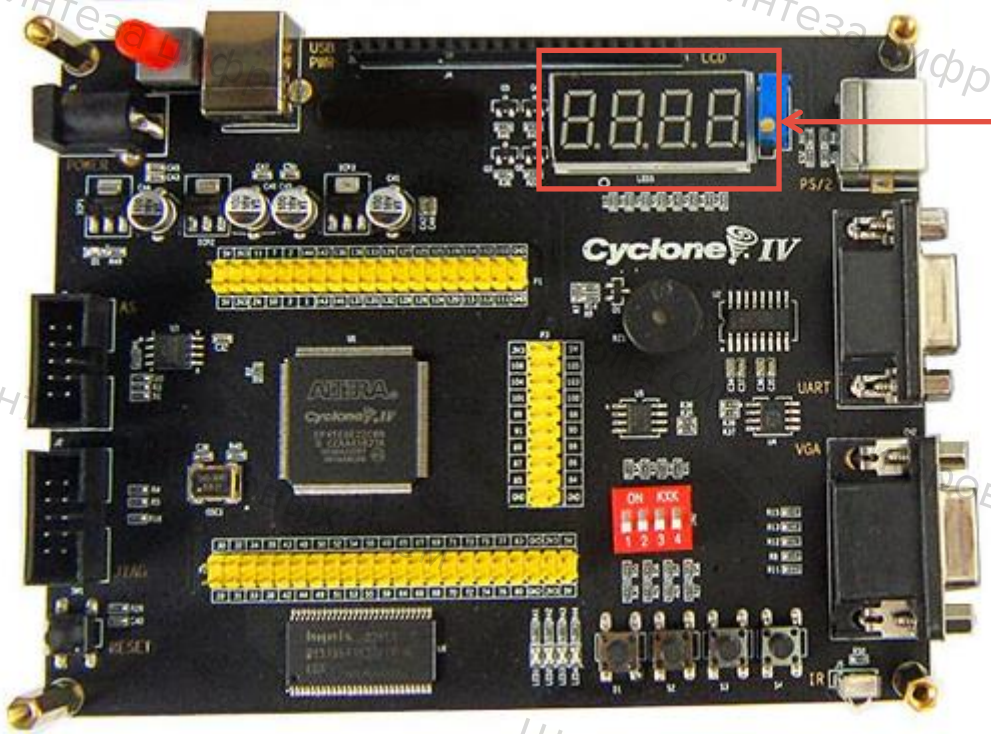
- Заставьте один огонек двигаться по кругу на светодиодах.
- Используйте другую кнопку, чтобы сбросить движущийся огонек обратно в режим отсутствия.

Упражнение 3

- Отобразите состояние сдвигового регистра на семисегментном дисплее, перемещая огонек по нему.

школа синтеза цифровых схем

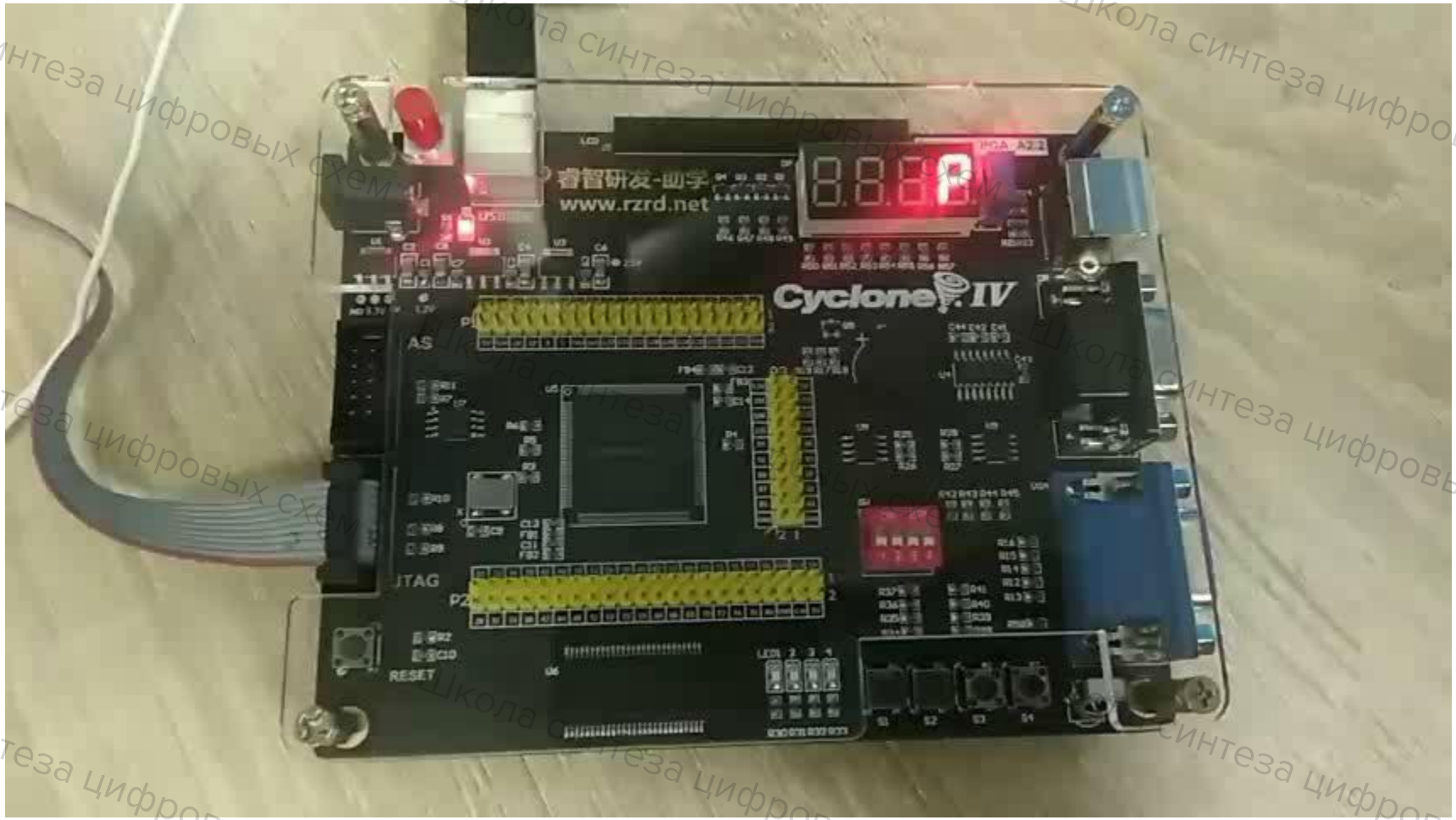
УПРАЖНЕНИЕ ВЫВОД СЛОВА НА СЕМИСЕГМЕНТНЫЙ ИНДИКАТОР 1_08_7SEGMENT_WORD



Вывод слова на семисегментный индикатор при помощи последовательностной логики.

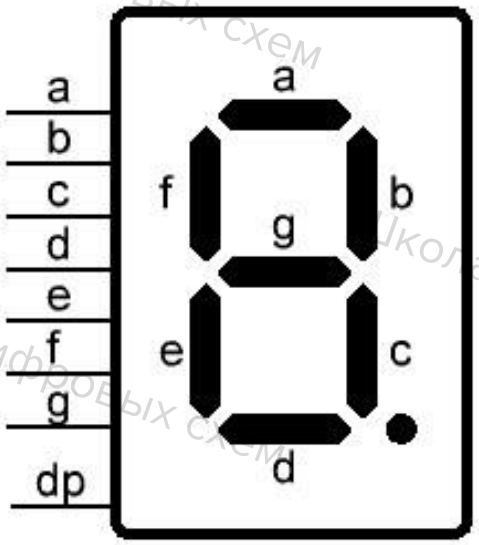
школа синтеза цифровых схем

УПРАЖНЕНИЕ ВЫВОД СЛОВА НА СЕМИСЕГМЕНТНЫЙ ИНДИКАТОР 1_08_7SEGMENT_WORD



СЕМИСЕГМЕНТНЫЙ ИНДИКАТОР

школа синтеза цифровых схем



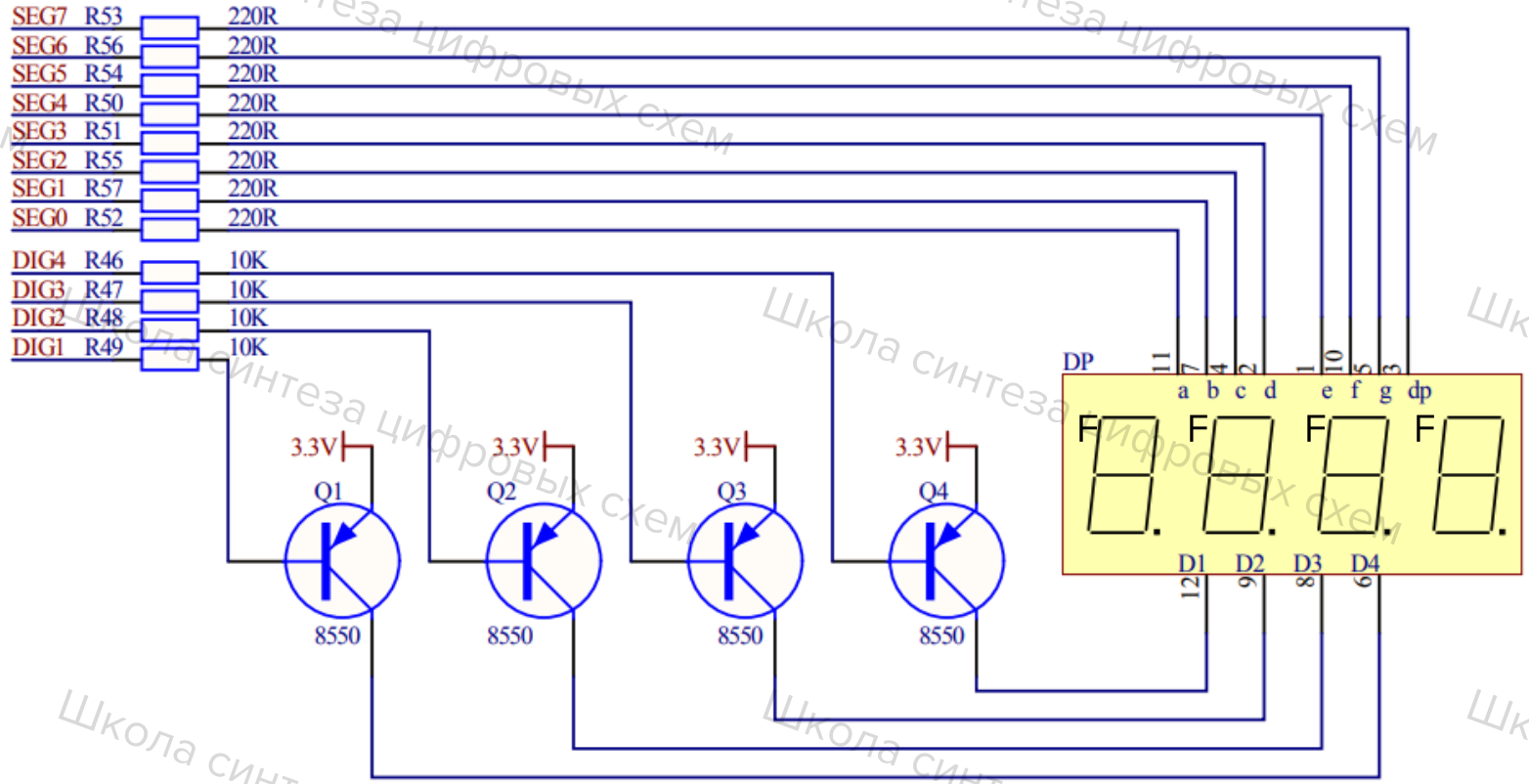
	4-bit шина				7-сегментный индикатор						
Dec	3	2	1	0	a	b	c	d	e	f	g
0	0	0	0	0	1	1	1	1	1	1	0
1	0	0	0	1	0	1	1	0	0	0	0
2	0	0	1	0	1	1	0	1	1	0	1
3	0	0	1	1	1	1	1	1	0	0	1
4	0	1	0	0	0	1	1	0	0	1	1
5	0	1	0	1	1	0	1	1	0	1	1
6	0	1	1	0	1	0	1	1	1	1	1
7	0	1	1	1	1	1	1	0	0	0	0
8	1	0	0	0	1	1	1	1	1	1	1
9	1	0	0	1	1	1	1	1	0	1	1

СЕМИСЕГМЕНТНЫЙ ИНДИКАТОР. ДИНАМИЧЕСКАЯ ИНДИКАЦИЯ

школа синтеза цифровых схем

F

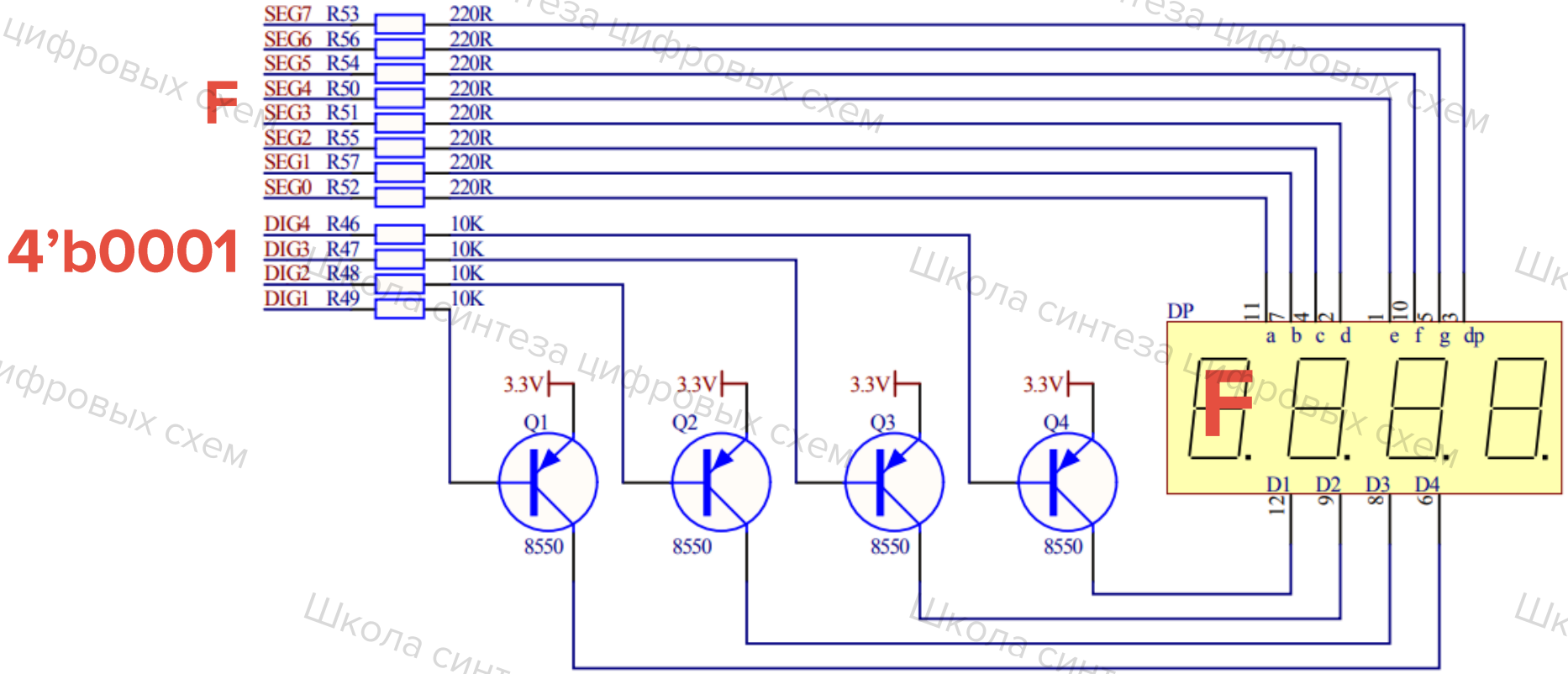
4’b1111



СЕМИСЕГМЕНТНЫЙ ИНДИКАТОР. ДИНАМИЧЕСКАЯ ИНДИКАЦИЯ

F

4’b0001

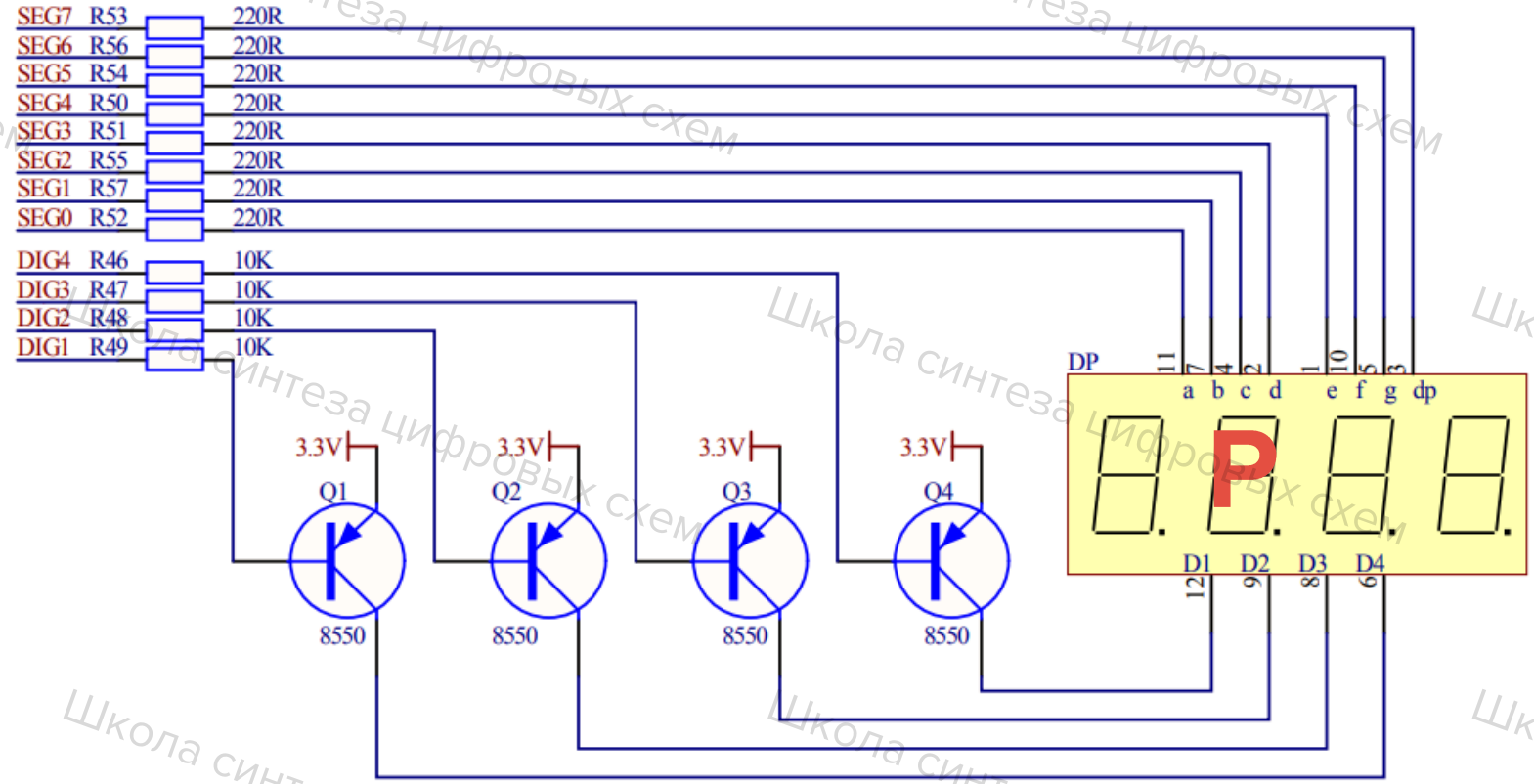


школа синтеза цифровых схем

СЕМИСЕГМЕНТНЫЙ ИНДИКАТОР. ДИНАМИЧЕСКАЯ ИНДИКАЦИЯ

P

4'b0010

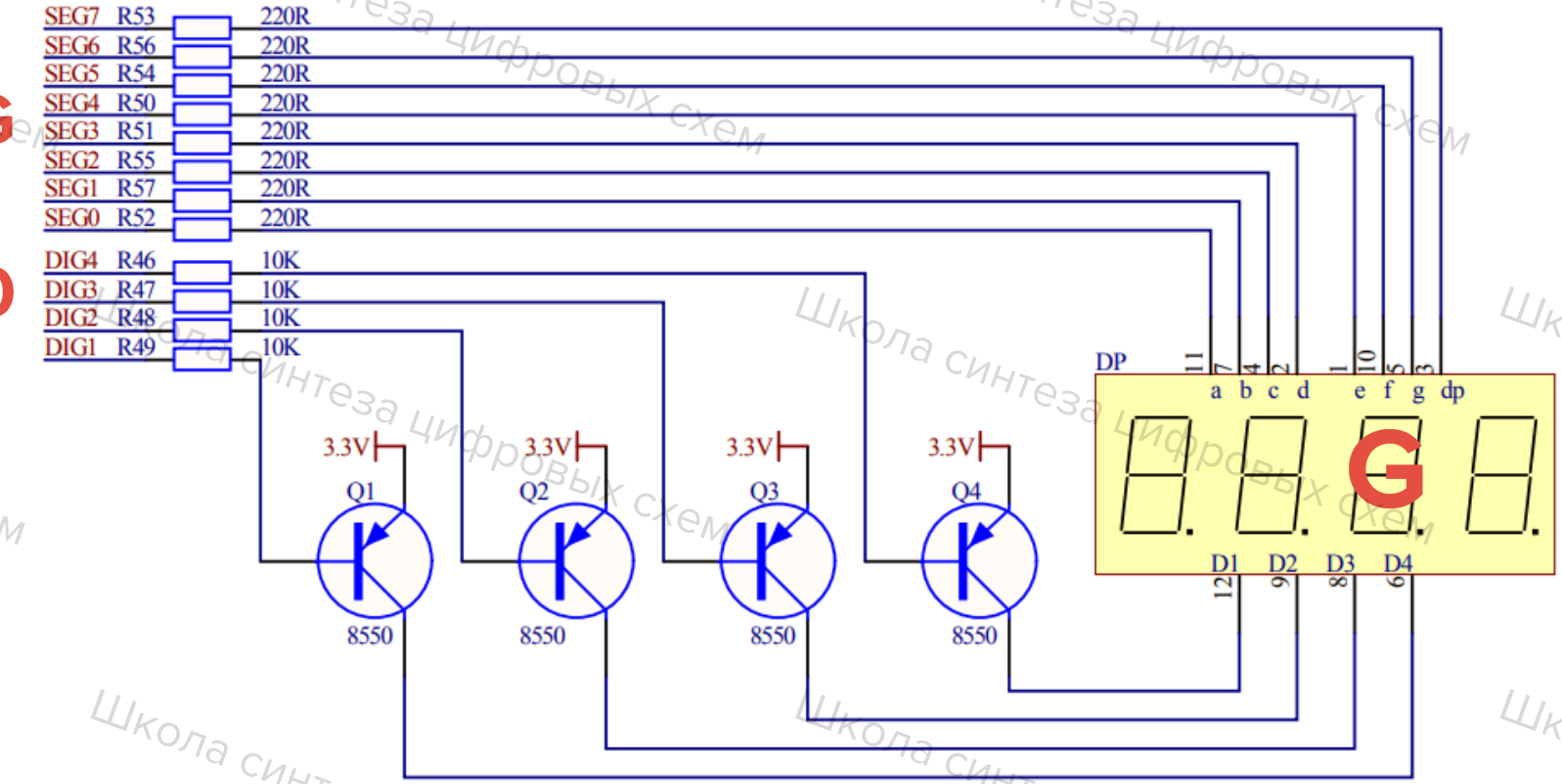


СЕМИСЕГМЕНТНЫЙ ИНДИКАТОР. ДИНАМИЧЕСКАЯ ИНДИКАЦИЯ

ШКОЛА СИНТЕЗА ЦИФРОВЫХ СХЕМ

G

4'b0100

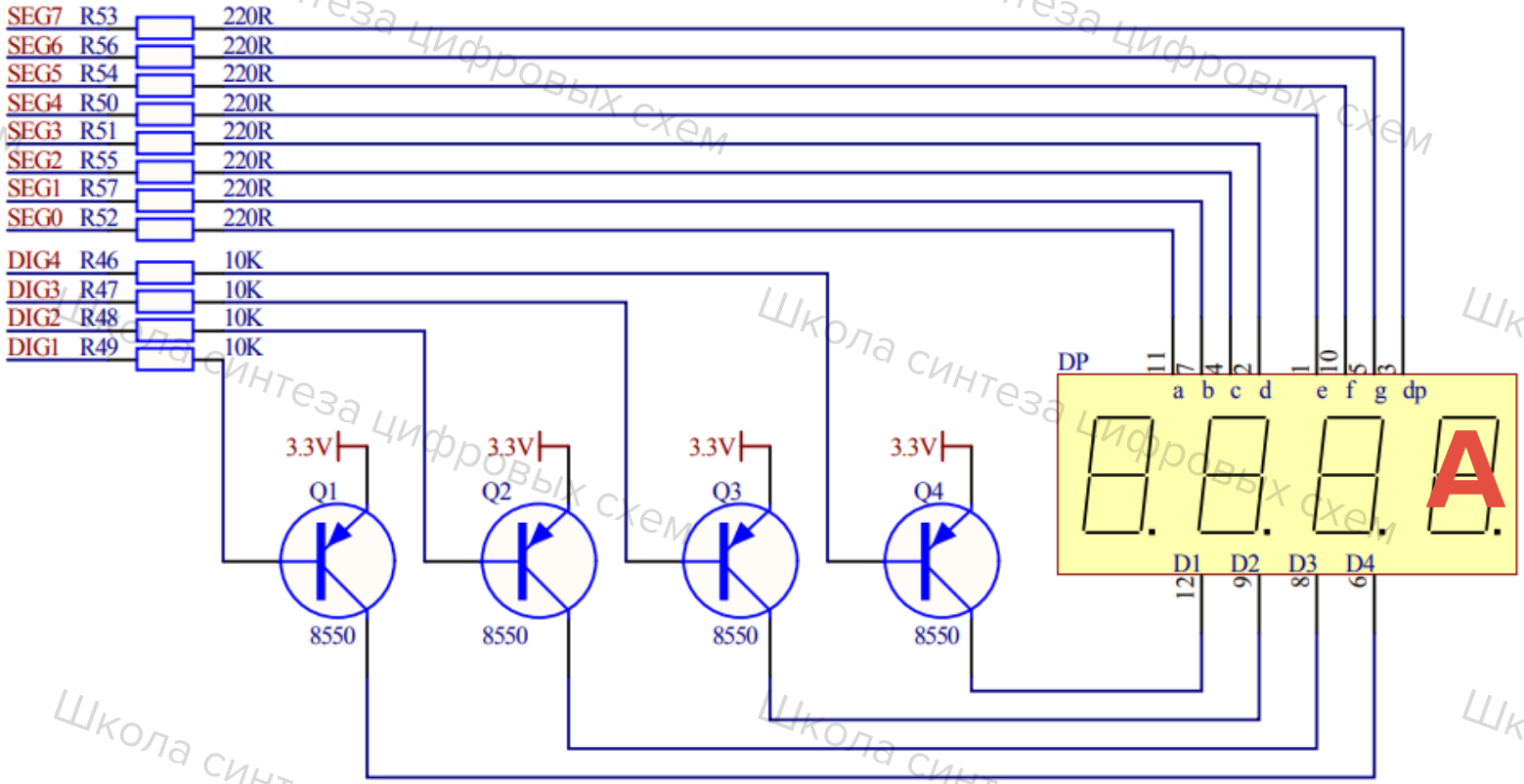


СЕМИСЕГМЕНТНЫЙ ИНДИКАТОР. ДИНАМИЧЕСКАЯ ИНДИКАЦИЯ

ШКОЛА СИНТЕЗА ЦИФРОВЫХ СХЕМ

A

4'b1000



школа синтеза цифровых схем

УПРАЖНЕНИЕ: ВЫВОД СЛОВА НА СЕМИСЕГМЕНТНЫЙ ИНДИКАТОР 1_08_7SEGMENT_WORD

```
logic [31:0] cnt;

always_ff @ (posedge clk or posedge rst)
    if (rst)
        cnt <= '0;
    else
        cnt <= cnt + 1'd1;

wire enable = (cnt [22:0] == '0);
```

Генерация сигнала enable который равен 1 один раз в 2**23 тактов. Изменив свариваемый диапазон, можно изменить частоту генерации сигнала enable.

Данный сигнал необходим для удобной визуализации работы сдвигового регистра в примере

школа синтеза цифровых схем

УПРАЖНЕНИЕ: ВЫВОД СЛОВА НА СЕМИСЕГМЕНТНЫЙ ИНДИКАТОР 1_08_7SEGMENT_WORD

```
logic [w_digit:0] shift_reg;

always_ff @ (posedge clk or posedge rst)
    if (rst)
        shift_reg <= w_digit' (1);
    else if (enable)
        shift_reg <= { shift_reg [0], shift_reg [w_digit - 1:1]
};

assign led = w_led' (shift_reg);
```

Циклический сдвиговый регистр, в котором по кругу бегает 1

0001 → 1000 → 0100 → 0010 → 0001

ШКОЛА СИНТЕЗА ЦИФРОВЫХ СХЕМ

УПРАЖНЕНИЕ: ВЫВОД СЛОВА НА СЕМИСЕГМЕНТНЫЙ ИНДИКАТОР 1_08_7SEGMENT_WORD

```
typedef enum bit [7:0]
{
    F     = 8'b1000_1110,
    P     = 8'b1100_1110,
    G     = 8'b1011_1100,
    A     = 8'b1110_1110,
    space = 8'b0000_0000
}
seven_seg_encoding_e;
```

```
seven_seg_encoding_e letter;

always_comb
   case (4' (shift_reg))
      4'b1000: letter = F;
      4'b0100: letter = P;
      4'b0010: letter = G;
      4'b0001: letter = A;
      default: letter = space;
   endcase

assign abcdefgh = letter;
assign digit    = shift_reg;
```

УПРАЖНЕНИЕ: ВЫВОД СЛОВА НА СЕМИСЕГМЕНТНЫЙ ИНДИКАТОР 1_08_7SEGMENT_WORD

Упражнение 1

- Увеличьте частоту сигнала enable до уровня, когда ваши глаза будут видеть буквы как сплошное слово без моргания. Каков порог такой частоты?

Упражнение 2

- Выведите свое имя или другое слово на дисплей.

Упражнение 3

- Закомментируйте строку "default" из оператора "case" в блоке "always" и заново синтезируйте пример. Получаете ли вы какие-либо предупреждения или ошибки?

Попробуйте объяснить почему.

Школа синтеза цифровых схем

УПРАЖНЕНИЕ 1_09_HEX_COUNTER

ШКОЛА СИНТЕЗА ЦИФРОВЫХ СХЕМ

```
// Exercise 1. Synthesize the counter controlled by two keys.
// When one key is in pressed position - the frequency increases,
// when another key is in pressed position - the frequency decreases.
// Change the period increment / decrement and see what happens.

logic [31:0] period;

localparam  min_period = clk_mhz * 1000 * 1000 / 30,
            max_period = clk_mhz * 1000 * 1000 *  3;
```

Школа синтеза цифровых схем

УПРАЖНЕНИЕ 1_09_HEX_COUNTER

ШКОЛА СИНТЕЗА ЦИФРОВЫХ СХЕМ

```
always_ff @ (posedge clk or posedge rst)
    if (rst)
        period <= 32' ((min_period + max_period) / 2);
    else if (key [0] & period != max_period)
        period <= period + 32'h1;
    else if (key [1] & period != min_period)
        period <= period - 32'h1;
1000 * 3;
```

Регистр, хранящий в себе настраиваемое значение счета cnt_1 (в некотором виде скорость счета). Обратите внимание что key[0] приоритетнее key[1]
А так же в условиях инкремента и декремента проверяется, что изменение счетчика не приведет к переполнению. Например, если period = 0 и пришло нажатие на key[1] то останется 0, а не будет значение ‘b11111

УПРАЖНЕНИЕ 1_09_HEX_COUNTER

ШКОЛА СИНТЕЗА ЦИФРОВЫХ СХЕМ

```
logic [31:0] cnt_1;

always_ff @ (posedge clk or posedge rst)
    if (rst)
        cnt_1 <= '0;
    else if (cnt_1 == '0)
        cnt_1 <= period - 1'b1;
    else
        cnt_1 <= cnt_1 - 1'd1;
```

Счетчик, отсчитывающий время в тактах от 0 до period -1. Изменяя значение period кнопками, можно изменить частоту, с которой cnt_1 обнуляется

Школа синтеза цифровых схем

УПРАЖНЕНИЕ 1_09_HEX_COUNTER

ШКОЛА СИНТЕЗА ЦИФРОВЫХ СХЕМ

```
logic [31:0] cnt_2;

always_ff @ (posedge clk or posedge rst)
    if (rst)
        cnt_2 <= '0;
    else if (cnt_1 == '0)
        cnt_2 <= cnt_2 + 1'd1;

assign led = cnt_2;
```

Счетчик, инкрементирующийся каждый раз, когда cnt_1 равен 0. Выведен на led в двоичном виде. Чем меньше period, тем чаще равен нулю cnt_1.
Тем быстрее меняется значение cnt_2

УПРАЖНЕНИЕ 1_09_HEX_COUNTER

ШКОЛА СИНТЕЗА ЦИФРОВЫХ СХЕМ

```
seven_seg_encoding_e letter;

always_comb
    case (4' (shift_reg))
        4'b1000: letter = F;
        4'b0100: letter = P;
        4'b0010: letter = G;
        4'b0001: letter = A;
        default: letter = space;
    endcase

assign abcdefgh = letter;
assign digit    = shift_reg;
```

```
// 4 bits per hexadecimal digit
localparam w_display_number = w_digit * 4;

seven_segment_display # (w_digit) i_7segment
(
    .clk      ( clk                          ),
    .rst      ( rst                          ),
    .number   ( w_display_number' (cnt_2)    ),
    .dots     ( w_digit' (0)                 ),
    .abcdefgh ( abcdefgh                     ),
    .digit    ( digit                        )
);
```

Реализуем дешифратор для семисегментного индикатора в виде отдельного модуля

УПРАЖНЕНИЕ 1_09_HEX_COUNTER

ШКОЛА СИНТЕЗА ЦИФРОВЫХ СХЕМ

```
// 4 bits per hexadecimal digit
localparam w_display_number = w_digit * 4;

seven_segment_display # (w_digit) i_7segment
(
    .clk      ( clk                          ),
    .rst      ( rst                          ),
    .number   ( w_display_number' (cnt_2) ),
    .dots     ( w_digit' (0)                 ),
    .abcdefgh ( abcdefgh                     ),
    .digit    ( digit                        )
);
```

Счетчик cnt_2 является 32-битным и будет выводиться на семисегментные индикаторы в виде 4 битный 16-ричных чисел. Каждый на отдельном индикаторе

УПРАЖНЕНИЕ: ВЫВОД СЛОВА НА СЕМИСЕГМЕНТНЫЙ ИНДИКАТОР 1_08_7SEGMENT_WORD

Упражнение 1

- Синтезируйте счетчик, управляемый двумя кнопками.

  Когда одна кнопка находится в нажатом положении — частота увеличивается, когда другая кнопка находится в нажатом положении — частота уменьшается.

  Измените шаг увеличения / уменьшения периода и посмотрите, что произойдет.

Упражнение 2

1. Удвоить частоту, когда одна кнопка нажимается и отжимается.
2. Уменьшите частоту вдвое, когда нажата и отжата вторая кнопка.

ПОКАЖИТЕ РЕШЕНИЕ КУРАТОРУ КЛАСТЕРА.